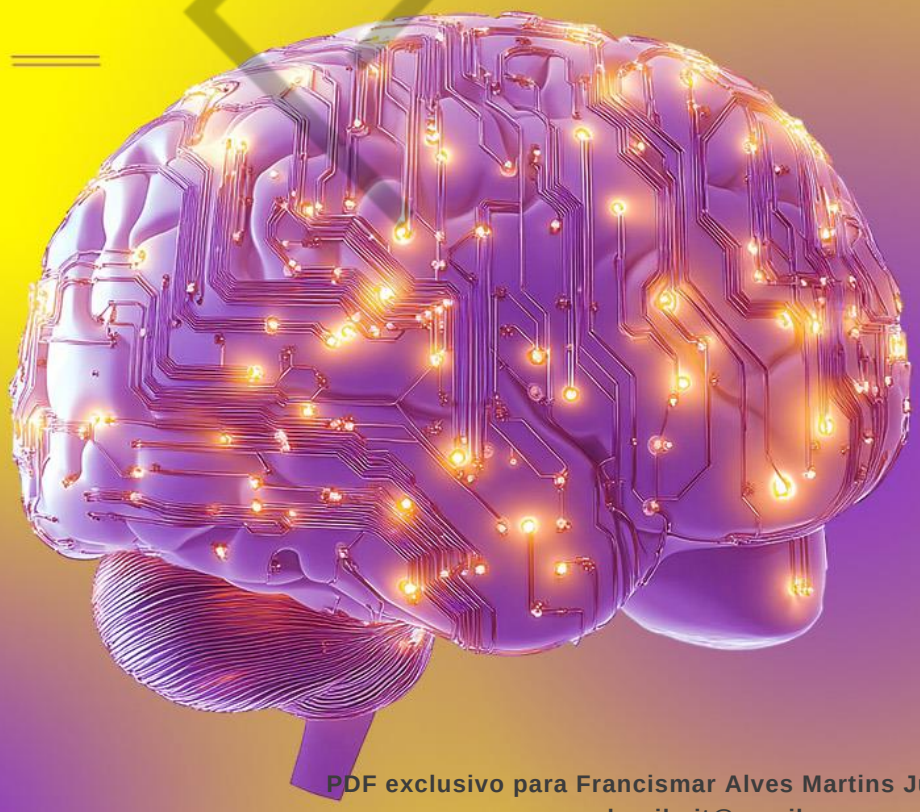
FIAP ON

IA COMO FERTILIZANTE DIGITAL - UM NOVO AGRONEGÓCIO DO AMANHÃ

APRENDENDO COM O TEMPO: O PODER DAS REDES NEURAIS RECORRENTES



2

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Imagem de uma bola em movimento em um instante qualquer.....7
Figura 2 - Imagens sequenciais de uma bola em movimento indicando sua trajetória 8
Figura 3 - Comparação da arquitetura de uma RNA com uma RNN.....10
Figura 4 - Representação de uma RNN desdobrada (*unfolded* RNN) 11
Figura 5 - Representação do problema de memória de curto prazo 13
Figura 6 - Representação de um neurônio de uma LSTM 15
Figura 7 - Representação de um neurônio de uma GRU 16
Figura 8 - Representação da estrutura geral de uma Rede Neural bidirecional.....17
Figura 9 - Representação da estrutura geral de uma rede neural Encoder-Decoder18
Figura 10 - Representação de séries temporais.....24
Figura 11 - Representação de tendências em séries temporais 26
Figura 12 - Representação de tendência e sazonalidade em série temporal.....27
Figura 13 - Representação de tendência e sazonalidade em série temporal.....27
Figura 14 - Exemplo de dados de série temporal.....31
Figura 15 - Exemplo de dados transformado para série temporal.....31
Figura 16 - Etapas do pipeline de tratamento de dados.....34
Figura 17 - Etapas do pipeline de tratamento de dados.....38
Figura 18 - Intuição do Word2Vec.....42
Figura 19 - Saída do Código-fonte 2 48
Figura 20 - Saída do Código-fonte 3 49
Figura 21 - Saída do Código-fonte 4 50
Figura 22 - Saída do Código-fonte 5 51
Figura 23 - Saída do Código-fonte 7 52
Figura 24 - Saída do Código-fonte 8 54
Figura 25 - Saída do Código-fonte 10 56
Figura 26 - Saída do Código-fonte 11 57
Figura 27 - Saída do Código-fonte 14 60
Figura 28 - Saída do Código-fonte 15 61
Figura 29 - Saída do Código-fonte 16 64
Figura 30 - Loss do treinamento.....66
Figura 31 - Diferenças entre os valores reais e preditos 66
Figura 32 - Tamanho do DataFrame 68
Figura 33 - Distribuição das classes na base 69
Figura 34 - Exemplos de e-mails da base fornecida 70
Figura 35 - Resultado do treinamento do modelo 75
Figura 36 - Matriz de confusão do melhor e pior classificador de e-mails.....77

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 1 - Vantagens e desvantagens do TF-IDF.....40

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Imports do código de LSTM....48
Código-fonte 2 - Carregando o dataset....48
Código-fonte 3 - Selecionando apenas a coluna “Open”....49
Código-fonte 4 - Plotando a série histórica dos dados....49
Código-fonte 5 - Padronizando a escala dos dados entre 0 e 1....50
Código-fonte 6 - Lógica para criar o dataset da série temporal....51
Código-fonte 7 - Reescalando o formato da variável de treino....52
Código-fonte 8 - Definindo, compilando e treinando a LSTM....53
Código-fonte 9 - Lendo e preparando os dados de teste....55
Código-fonte 10 - Predizendo ações a partir da série temporal....55
Código-fonte 11 - Predizendo ações a partir da série temporal....56
Código-fonte 12 - Baixando os dados brutos da predição de poluição....59
Código-fonte 13 - Adaptando os dados da série temporal....59
Código-fonte 14 - Mostrando as primeiras linhas....59
Código-fonte 15 - Explorando algumas variáveis....60
Código-fonte 16 - Lendo e preparando os dados para a análise multivariada....63
Código-fonte 17 - Treinando e avaliando o modelo preditivo....65
Código-fonte 18 - Imports e leitura da base....68
Código-fonte 19 - Analisando a distribuição da base....68
Código-fonte 20 - Inspecionando alguns e-mails....69
Código-fonte 21 - Limpando o texto....70
Código-fonte 22 - Dividindo a base em TREINO e TESTE....71
Código-fonte 23 - Criando as representações numéricas dos textos....72
Código-fonte 24 - Definindo e treinando os modelos preditivos....73
Código-fonte 25 - Avaliando o desempenho dos modelos....74
Código-fonte 26 - Comparando o melhor e o pior modelo....76

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 APRENDENDO COM O TEMPO: O PODER DAS REDES NEURAIS RECORRENTES 7
1.1 Por que usar Redes Neurais Recorrentes? 7
1.2 Aplicações 8

2 ARQUITETURAS DE REDES NEURAIS RECORRENTES 10
2.1 Diferença entre redes feedforward e recorrentes 10
2.2 Por que RNNs falham em sequências longas? 11
2.3 O problema do desaparecimento do gradiente 12
2.4 Arquiteturas derivadas das Redes Neurais Recorrentes 14
2.4.1 LSTM 15
2.4.2 GRU 16
2.4.3 Bidirectional RNN 17
2.4.4 Encoder-Decoder 17

3 ARQUITETURA LSTM 19
3.1 Visão geral 19
3.2 Componentes das LSTMs 19
3.3 Diferença entre GRU e LSTM 21
3.4 Fluxo de informações de uma célula LSTM 21
3.5 Melhores práticas 22

4 PREDIÇÃO DE SÉRIES TEMPORAIS 24
4.1 Aplicações 24
4.2 Características desejadas nos dados 25
4.3 Preparação dos dados 28
4.3.1 Alterando a série temporal para atingir as características desejadas 29
4.3.2 Ajuste no formato da base de dados 30
4.3.3 Estratégias para divisão dos dados entre treino e teste 32
4.4 Séries univariadas e multivariadas 33

5 PROCESSAMENTO DE LINGUAGEM NATURAL 34
5.1 Limpeza 35
5.2 Representação de texto 37
5.2.1 Bag of Words (BoW) 37
5.2.2 Term Frequency-Inverse Document Frequency (TF-IDF) 39
5.2.3 Embeddings de palavras 41

6 TRANSFORMERS *VERSUS* RNNS: O DECLÍNIO DAS LSTMS? 44
6.1 Por que os transformers substituíram as RNNs em NLP? 44
6.2 Diferenças fundamentais: atenção *versus* recorrência 44
6.2.1 Transformers 45
6.2.2 BERT 45

7 HANDS ON: APLICAÇÕES DE SÉRIES TEMPORAIS 47
7.1 Predição de ações (série univariada) 47
7.2 Predição de poluição do ar (série multivariada) 57
7.3 Classificação de e-mails em spam ou não (trabalhando com texto) 67

CONCLUSÃO 78

REFERÊNCIAS 79

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO 81

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 APRENDENDO COM O TEMPO: O PODER DAS REDES NEURAIS RECORRENTES

1.1 Por que usar Redes Neurais Recorrentes?

As Redes Neurais Recorrentes (RNNs) são um tipo especial de Rede Neural Artificial desenvolvida para lidar com dados sequenciais, ou seja, informações organizadas em uma sequência onde a ordem dos elementos importa. Isso significa que, ao contrário de redes neurais tradicionais, que processam cada entrada de forma independente, as RNNs possuem um mecanismo de memória que lhes permite considerar eventos anteriores ao fazer previsões ou classificações.

Para entender a importância das RNNs, considere uma cena onde há uma bola em movimento. Se você visse apenas uma foto estática da bola, como na figura a seguir, seria difícil prever sua trajetória ou para onde ela está indo, pois existem infinitas possibilidades. Observe:

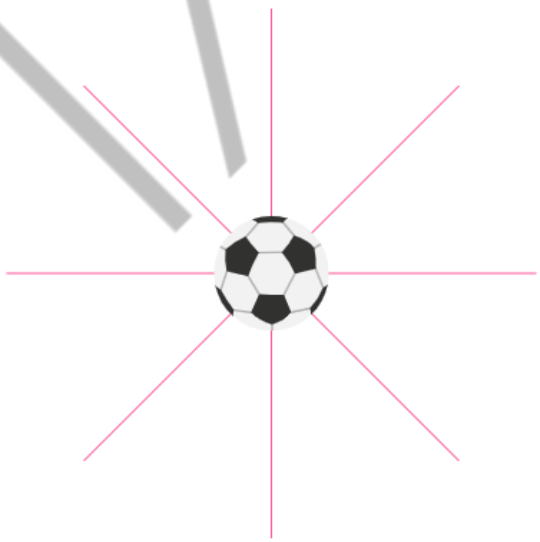


Figura 1 - Imagem de uma bola em movimento em um instante qualquer
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

No entanto, se tivéssemos uma sequência de fotos tiradas em momentos diferentes, poderíamos observar a direção da bola ao longo do tempo, permitindo-nos fazer uma previsão mais precisa sobre seu movimento futuro, como mostra a próxima figura.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 2 - Imagens sequenciais de uma bola em movimento indicando sua trajetória
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

Da mesma forma, as RNNs são capazes de lidar com dados sequenciais, capturando a dependência temporal entre os elementos da sequência.

Ao contrário das redes neurais convolucionais (CNNs), que são mais adequadas para dados com estrutura espacial, como imagens, **as RNNs são projetadas para lidar com dados sequenciais**, como texto, áudio, entre outros. Essa capacidade de modelar dependências temporais as torna extremamente poderosas em uma variedade de aplicações, incluindo novas formas de interação entre humanos e máquinas, algumas análises típicas da Saúde e do setor Financeiro, bem como predição de clima.

A principal característica distintiva das RNNs é sua capacidade de manter estados anteriores e usar essas informações para processar dados subsequentes na sequência. Isso permite que as RNNs capturem dependências de longo alcance em dados sequenciais, o que é crucial em muitas tarefas de Aprendizado de Máquina.

No entanto, as RNNs também enfrentam desafios, como a própria forma de organizar e estruturar um banco de dados dependente do tempo. Além disso, a necessidade de memorizar sequências longas implica em maior consumo de memória e dados. Para superar esses obstáculos, foram desenvolvidas variantes das RNNs, como as redes neurais recorrentes de memória de curto e longo prazo (LSTMs), que veremos adiante.

1.2 Aplicações

As RNNs têm se destacado em uma variedade de aplicações de negócios, oferecendo soluções inovadoras e eficazes para uma série de desafios. Uma das áreas mais proeminentes é o **reconhecimento de fala**, onde as RNNs são empregadas para transcrever áudio em texto com alta precisão. Empresas de tecnologia como Google, Amazon e Apple têm investido pesadamente em sistemas

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

de reconhecimento de fala baseados em RNNs, o que tem melhorado significativamente a usabilidade de seus assistentes virtuais e dispositivos inteligentes.

Além disso, as RNNs são amplamente utilizadas na **tradução automática**, facilitando a comunicação entre pessoas que falam idiomas diferentes. Plataformas como o Google Translate empregam modelos de tradução baseados em RNNs para oferecer traduções rápidas e precisas em uma ampla variedade de idiomas. Essa aplicação não apenas torna a comunicação global mais acessível, mas abre novas oportunidades de mercado para empresas buscando expandir a presença internacional.

Outro campo de aplicação importante é a **previsão de séries temporais**, onde as RNNs têm se destacado no fornecimento de insights valiosos para empresas de diversos setores. Em finanças, por exemplo, as RNNs são usadas para **prever preços de ações**, permitindo que investidores tomem decisões mais informadas e estratégicas. Da mesma forma, em empresas de varejo, as RNNs são empregadas para **prever demanda de produtos**, otimizando o estoque e reduzindo custos operacionais.

Além dessas áreas, as RNNs encontram aplicação em uma variedade de outros contextos de negócios. Por exemplo, no marketing digital, as RNNs são usadas para **personalizar recomendações de produtos com base no histórico de compras** dos clientes, aumentando a taxa de conversão e o engajamento do usuário. Em medicina, as RNNs são aplicadas para **análise de séries temporais de sinais vitais**, auxiliando médicos no diagnóstico precoce de doenças e na tomada de decisões clínicas mais precisas.

Em resumo, as RNNs oferecem um arsenal versátil e poderoso para solucionar uma variedade de problemas de negócios, desde melhorar a experiência do usuário até otimizar processos operacionais e informar decisões estratégicas. Com sua capacidade de lidar com dados sequenciais e capturar dependências temporais complexas, as RNNs estão transformando a forma como as empresas abordam os desafios do mundo real, impulsionando a inovação e o sucesso comercial.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 ARQUITETURAS DE REDES NEURAIS RECORRENTES

2.1 Diferença entre redes feedforward e recorrentes

A estrutura básica de uma RNN consiste em unidades recorrentes conectadas em série, formando loops que permitem que as informações sejam propagadas ao longo da sequência temporal. Visualmente, uma RNN pode ser representada como uma série de blocos conectados, onde cada bloco representa uma unidade recorrente e as conexões entre os blocos indicam a passagem das informações ao longo do tempo.

A próxima figura compara a representação de uma arquitetura de Rede Neural Artificial Feedforward (a) com de uma Rede Neural Recorrente (b):

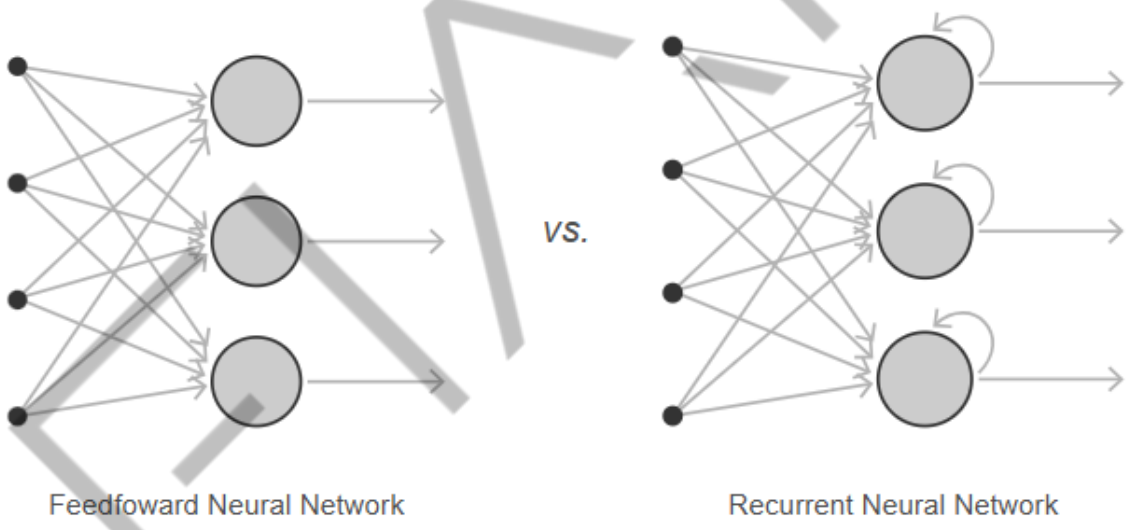


Figura 3 - Comparação da arquitetura de uma RNA com uma RNN
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

A figura “Comparação da arquitetura de uma RNA com uma RNN” mostra que ao contrário de uma Rede Neural comum, onde cada entrada é tratada de forma independente, uma RNN processa sequências de dados, mantendo uma memória interna que captura as dependências temporais. Esse processo permite que a rede faça previsões ou classificações com base em informações contextuais, levando em consideração o histórico da sequência.

Uma RNN tradicional opera em um único passo de tempo de cada vez, o que pode ocasionar problemas de dissipação ou explosão do gradiente durante o

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

treinamento. Para mitigar esses problemas, as RNNs desdobradas (*unfolded* RNNs) foram introduzidas. Observe a figura:

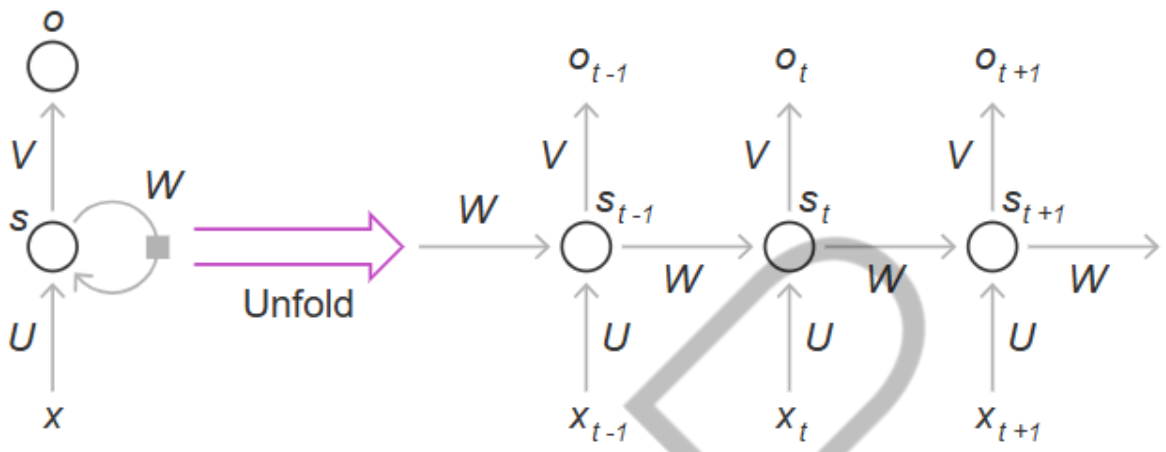


Figura 4 - Representação de uma RNN desdobrada (*unfolded* RNN)
Fonte: ResearchGate (2018), adaptado por FIAP (2025)

Nessa abordagem, a rede é expandida em múltiplos passos de tempo, criando uma representação gráfica que se estende ao longo da sequência temporal. Isso permite que as informações fluem livremente ao longo da rede, facilitando o treinamento e permitindo que a rede capture relações de longo prazo entre os dados. Durante o treinamento, as RNNs são atualizadas usando o algoritmo de retropropagação no tempo (Backpropagation Through Time — BPTT), que é uma extensão do algoritmo de retropropagação utilizado em redes neurais comuns. Esse processo ajusta os pesos da rede de acordo com o erro calculado em cada passo de tempo, permitindo que a rede aprenda a modelar dependências temporais nos dados de entrada.

2.2 Por que RNNs falham em sequências longas?

As RNNs apresentam diversas **forças** que as tornam poderosas em lidar com dados sequenciais. Uma delas é a capacidade de processar entradas de qualquer tamanho, permitindo flexibilidade no tamanho das sequências de dados que podem ser alimentadas na rede. Além disso, o tamanho do modelo não aumenta com o tamanho da entrada, isso significa que a mesma arquitetura pode ser aplicada a diferentes tamanhos de sequência, economizando recursos computacionais.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Outra vantagem significativa das RNNs é a capacidade de considerar informações históricas. Isso significa que a computação, em um determinado passo de tempo, leva em consideração o contexto das etapas anteriores, permitindo que a rede capture dependências temporais nos dados. Além disso, o compartilhamento de pesos ao longo do tempo (presente em arquiteturas derivadas das RNNs) ajuda a reduzir o número de parâmetros do modelo, tornando-o mais eficiente e eficaz em aprender representações temporais.

No entanto, as RNNs também apresentam algumas **fraquezas** importantes. Uma delas é a computação lenta, especialmente em sequências longas, devido à natureza serial do processamento. Além disso, as RNNs podem ter dificuldade em acessar informações de muito tempo atrás devido à limitada capacidade de memória, muitas vezes referida como “memória curta”.

Por fim, as RNNs não podem considerar informações futuras para o estado atual, e a incorporação de dados externos pode ser complexa, o que limita sua capacidade de modelar relações complexas em dados sequenciais.

2.3 O problema do desaparecimento do gradiente

Entre os diversos desafios de trabalhar com RNNs, o principal é o **desaparecimento do gradiente**. Para entender este fenômeno, primeiramente devemos compreender o conceito de “**memória sequencial**”, que é a base de como as RNNs aprendem. Toda RNN está baseada no princípio da memória sequencial.

A **memória sequencial** refere-se à capacidade de um sistema de armazenar e acessar informações de eventos anteriores em uma sequência de dados. Nós também temos esta memória! Para ilustrarmos isso, faça o seguinte exercício:

- **Passo 1:** diga o alfabeto na ordem comum;
- **Passo 2:** imagino que o primeiro passo foi fácil! Agora diga o alfabeto na ordem inversa;
- **Passo 3:** acredito que o segundo passo foi bem mais complicado! Então diga o alfabeto a partir da letra “F”;

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Passo 4:** imagino que você precisou pensar um pouco para chegar à letra “F”, depois as próximas letras vieram naturalmente na sua mente.

A memória sequencial nos ajuda a recuperar informações em ordens esperadas, o que confere velocidade nas respostas. Outro exemplo simples é a previsão do próximo caractere em uma sequência de palavras. Por exemplo, ao tentar prever a próxima letra da palavra "*hello*", saber que a letra "h" é seguida por "e", que por sua vez é seguida por "l", é crucial para prever com precisão que a próxima letra é "l".

A capacidade conferida pela memória sequencial é relevante para as RNNs porque viabiliza a captura de dependências nos dados, que podem ser exploradas pela rede para predições futuras.

O problema ocorre quando para compreender um contexto tentamos (ou precisamos) acumular muita informação, ou, ainda, necessitamos de informação mais antiga. A figura a seguir ilustra essa questão.

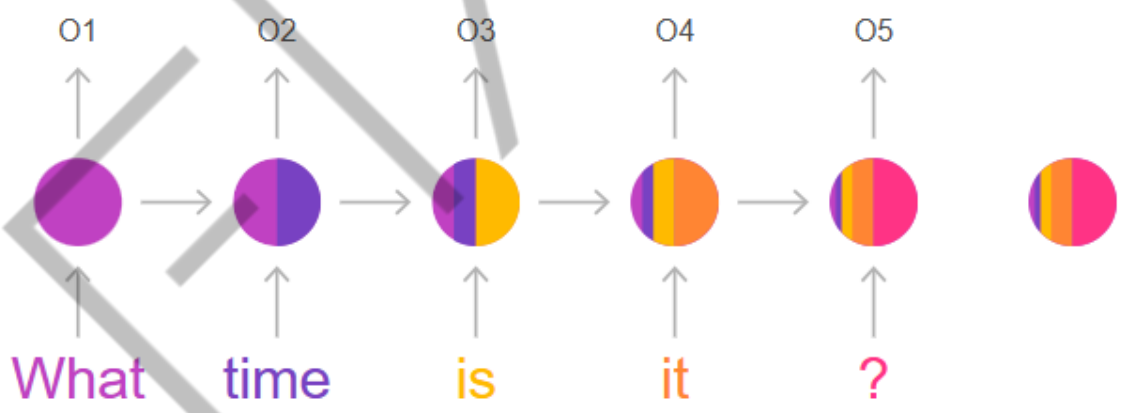


Figura 5 - Representação do problema de memória de curto prazo
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

A figura “Representação do problema de memória de curto prazo” mostra, de forma simplificada, o acúmulo de informação em um neurônio recorrente para aprender a decifrar o conteúdo da frase “*What time is it?*” (“Que horas são?” — em inglês). Por questões didáticas vamos manter o sinal de interrogação na frase. Em um pipeline tradicional de Processamento de Linguagem Natural (PLN) qualquer pontuação seria removida. Na primeira interação, o neurônio “aprende” apenas o primeiro item “what”, cuja informação é parcialmente retida e utilizada na próxima

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

interação, aprendendo a combinação “what time”. Esta combinação, por sua vez, influencia o aprendizado do próximo item “is”, e assim sucessivamente.

À cada iteração, notamos que o aprendizado acumulado representa aproximadamente 50% da informação retida, e que os outros 50% vêm da informação atual, independentemente de qual seja.

O destaque à direita da figura “Representação do problema de memória de curto prazo” ilustra “o aprendizado acumulado” do neurônio em questão ao final do processamento. Note que ~75% do que é “mais importante” para ele é o dado mais recente “is it?”, sendo que o sentido da frase está justamente nos primeiros itens “What time”, que representam ~15% do conteúdo retido.

A memória sequencial leva ao problema da memória de curto prazo, o que gera o **problema do desaparecimento do gradiente**. Isto é uma dificuldade encontrada durante o treinamento de redes neurais profundas, onde os gradientes associados aos pesos das camadas anteriores da rede se tornam cada vez menores à medida que são propagados para trás durante o processo de retropropagação. Esse problema surge devido à multiplicação contínua de gradientes menores que 1 ao longo das camadas da rede, fazendo com que os gradientes diminuam exponencialmente à medida que retrocedem nas camadas.

Esse fenômeno é problemático porque os gradientes, que são muito pequenos, podem ocasionar um treinamento ineficiente ou até mesmo a estagnação do processo de atualização dos pesos.

2.4 Arquiteturas derivadas das Redes Neurais Recorrentes

Não podemos negar o sucesso e a importância das RNNs. Então, para amenizar os problemas enfrentados, outras arquiteturas de redes recorrentes foram propostas, cada uma com suas características distintas e aplicações específicas. Vejamos as principais:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.4.1 LSTM

Assim como a RNN padrão, a LSTM é projetada para processar sequências temporais e capturar dependências de longo prazo. No entanto, a LSTM possui uma arquitetura mais complexa, introduzindo unidades de memória que ajudam a superar o problema do desaparecimento do gradiente. Ela é composta por três portas principais (entrada, esquecimento e saída) que controlam o fluxo de informação ao longo do tempo, conforme ilustrado na figura “Representação de um neurônio de uma LSTM”.

- **Pontos fortes:** é altamente eficaz em capturar dependências de longo prazo em sequências temporais, mantendo a capacidade de aprender dependências de curto prazo.
- **Pontos fracos:** pode ser mais computacionalmente intensiva comparado a RNN padrão devido à sua arquitetura mais complexa, o que pode resultar em tempos de treinamento mais longos.
- **Aplicações principais:** tradução automática, geração de legendas de imagens, análise de sentimento em texto.

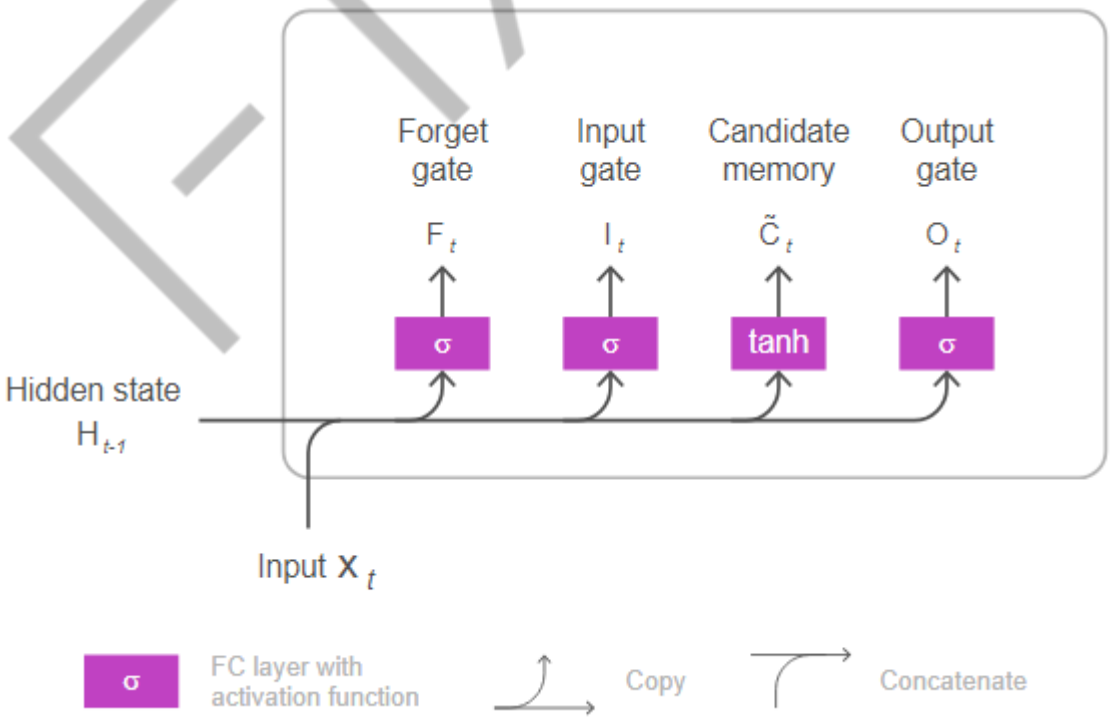


Figura 6 - Representação de um neurônio de uma LSTM
Fonte: Dive into Deep Learning (2025), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.4.2 GRU

A GRU é uma variação da RNN que também opera em sequências temporais e é capaz de lidar com dependências de longo prazo. No entanto, possui uma estrutura mais simples do que a LSTM, com menos portas de controle de fluxo de informação. Ela combina as funções de atualização e esquecimento em uma única porta, tornando-a computacionalmente mais eficiente, como podemos observar na figura “Representação de um neurônio de uma GRU”.

- **Pontos fortes:** a GRU é eficaz em capturar dependências de longo prazo em sequências temporais, com menor complexidade computacional em comparação com a LSTM.
- **Pontos fracos:** como possui menos portas de controle do que a LSTM, a GRU pode ser menos expressiva em algumas tarefas que exigem modelagem de dependências complexas de longo prazo.
- **Aplicações principais:** reconhecimento de fala, tradução automática, análise de sentimentos em texto.

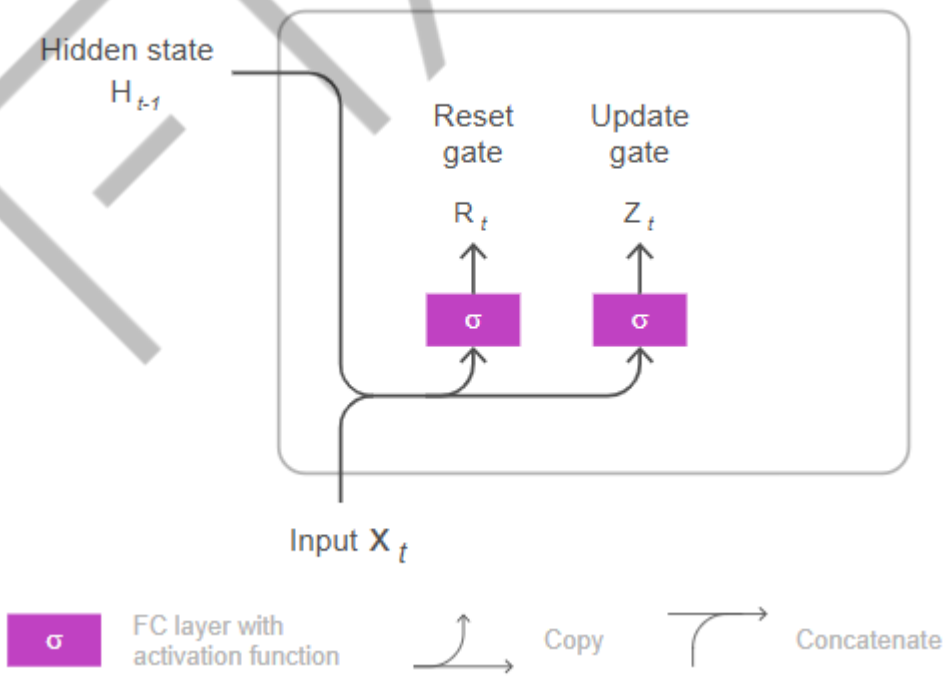


Figura 7 - Representação de um neurônio de uma GRU
Fonte: Dive into Deep Learning (2025), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.4.3 Bidirectional RNN

Assim como a RNN padrão, a Bidirectional RNN opera em sequências temporais, mas com a capacidade de considerar informações passadas e futuras simultaneamente. A Bidirectional RNN consiste em duas camadas de células RNN, uma processando a sequência temporal no sentido direto e a outra processando no sentido reverso. Isso permite que a rede capture informações contextuais de ambos os lados da sequência, conforme ilustrado na figura “Representação da estrutura geral de uma Rede Neural bidirecional”.

- **Pontos fortes:** a Bidirectional RNN é eficaz em capturar contexto global em sequências temporais, levando em conta tanto o passado quanto o futuro.
- **Pontos fracos:** pode ser mais computacionalmente intensiva do que a RNN padrão devido à necessidade de processar a sequência em ambos os sentidos.
- **Aplicações principais:** reconhecimento de entidades nomeadas em linguagem natural, segmentação de fala em texto.

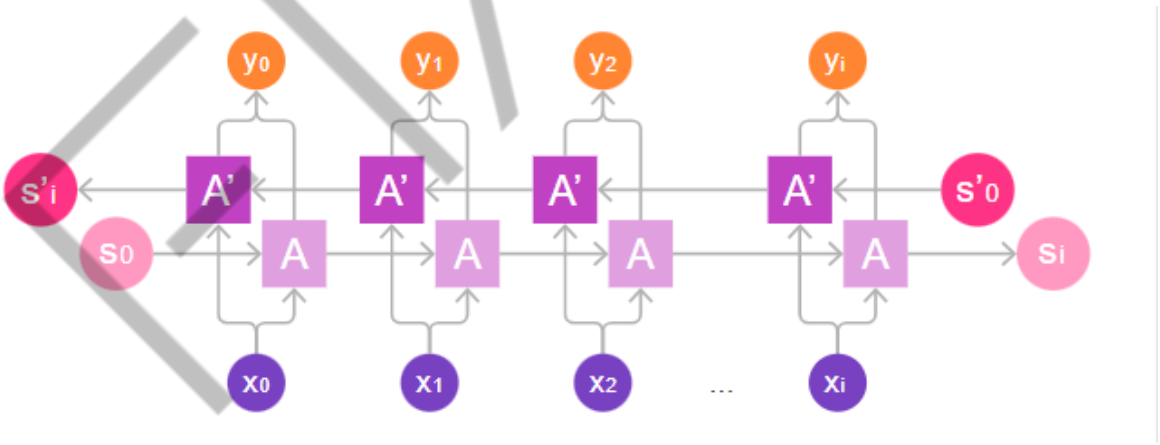


Figura 8 - Representação da estrutura geral de uma Rede Neural bidirecional
Fonte: ResearchGate (2021), adaptado por FIAP (2025)

2.4.4 Encoder-Decoder

O Encoder-Decoder é uma estrutura composta por duas redes RNNs, uma atuando como codificador e outra como decodificador, que operam em sequências temporais. É projetado para lidar com tarefas de sequência para sequência, como tradução automática, onde uma sequência de entrada é codificada em um vetor de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

contexto e, em seguida, decodificada para uma sequência de saída em outra língua (outra representação). Esse conceito está ilustrado na figura “Representação da estrutura geral de uma rede neural Encoder-Decoder”.

- **Pontos fortes:** é altamente eficaz em tarefas de geração de sequência para sequência, como tradução automática e resumo de texto.
- **Pontos fracos:** pode ser desafiador treinar efetivamente em conjuntos de dados grandes devido à necessidade de capturar informações contextuais de longo prazo.
- **Aplicações principais:** tradução automática, geração de legendas de imagens, resumo automático de texto.

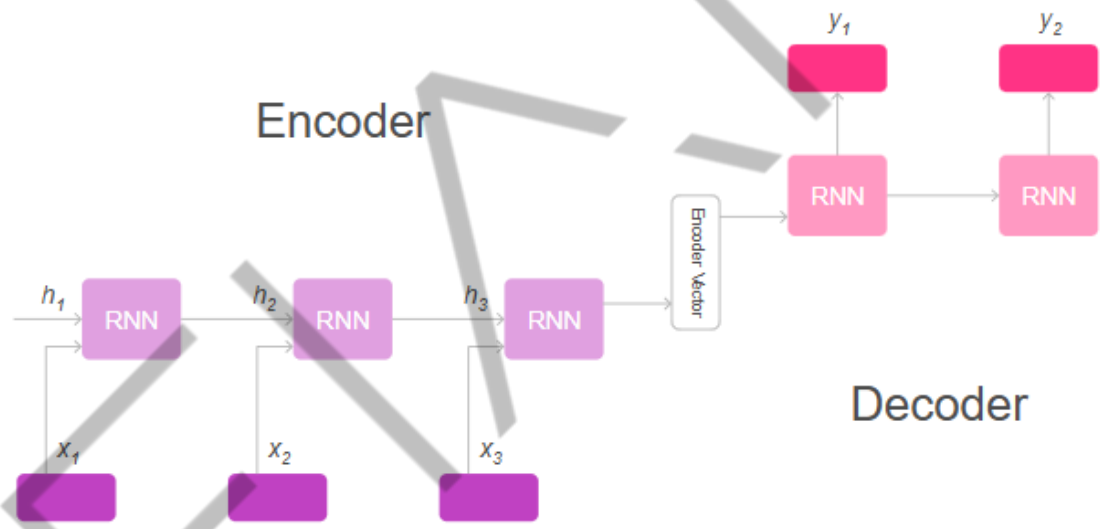


Figura 9 - Representação da estrutura geral de uma rede neural Encoder-Decoder
Fonte: Medium (2021), adaptado por FIAP (2025)

Essas são apenas algumas das arquiteturas derivadas das RNNs que ampliam suas capacidades e aplicações em várias áreas, oferecendo soluções para problemas específicos encontrados no processamento de sequências temporais. Cada uma dessas arquiteturas tem suas vantagens e limitações, e vale lembrar que a escolha entre elas depende da natureza da tarefa e dos requisitos específicos do problema em questão.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 ARQUITETURA LSTM

3.1 Visão geral

Entre as arquiteturas derivadas das RNNs, as LSTMs possuem destaque pela sua grande variedade de aplicações e por endereçar o problema de desaparecimento do gradiente.

Desenvolvida por Hochreiter e Schmidhuber em 1997, a LSTM foi projetada para capturar dependências temporais de longo prazo, preservando informações importantes ao longo de muitos passos no tempo. Ela introduz unidades de memória controladas por portas, que permitem que a rede aprenda quais informações reter e quais descartar durante o processo de treinamento. Isso a torna particularmente eficaz em tarefas que exigem a modelagem de dependências temporais complexas, como tradução automática, geração de legendas de imagens e análise de séries temporais.

Nos últimos anos, houve avanços significativos na arquitetura e no treinamento de LSTMs, impulsionados principalmente pelo aumento do poder computacional e pelo desenvolvimento de técnicas de otimização mais eficazes. Pesquisadores têm explorado variações e melhorias na estrutura básica da LSTM, além de estratégias de treinamento mais avançadas, como o uso de algoritmos de otimização adaptativa e técnicas de regularização mais sofisticadas. Essas evoluções têm resultado em melhorias significativas no desempenho e na eficiência das LSTMs, tornando-as uma escolha cada vez mais popular em uma ampla gama de aplicações de Aprendizado de Máquina.

3.2 Componentes das LSTMs

As LSTMs continuam sendo RNNs formadas por neurônios organizados em camadas conectadas. A grande diferença está em como os neurônios são formados. Nas LSTMs cada célula neural é mais complexa, consistindo em quatro componentes principais: **unidades de memória**, **portões de entrada**, **portões de esquecimento** e **portões de saída**. A importância de cada componente é explicada a seguir:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Unidades de memória:** as unidades de memória em uma LSTM são responsáveis por **armazenar e atualizar informações** ao longo do tempo. Elas mantêm um estado de memória contínuo, que pode ser modificado por meio dos portões de entrada e de esquecimento.
- **Portões de Entrada (*Input Gates*):** os portões de entrada controlam a quantidade de **nova informação que será adicionada** à memória atual. Eles decidem quais valores serão atualizados na memória com base na entrada atual e no estado anterior.
- **Portões de Esquecimento (*Forget Gates*):** os portões de esquecimento determinam **quais informações antigas na memória serão descartadas**. Eles são responsáveis por decidir o que manter e o que esquecer, com base na relevância das informações para a tarefa atual.
- **Portões de Saída (*Output Gates*):** os portões de saída regulam a **quantidade de informação que será transmitida para a próxima camada** da rede ou para a saída final. Eles decidem qual parte da memória atual será exposta como saída da célula LSTM.

Para programar e configurar uma célula LSTM, é necessário definir os pesos associados a cada um dos componentes mencionados acima. Esses pesos são ajustados durante o treinamento da rede por meio do algoritmo de retropropagação pelo tempo, que calcula os gradientes dos pesos em relação à função de perda ao longo de uma sequência de entrada. Durante o treinamento, o *backpropagation* temporal é responsável por propagar o erro ao longo do tempo e ajustar os pesos de cada componente da célula LSTM para minimizar a perda.

Os componentes de uma célula LSTM são altamente influenciados pelo BPTT, pois suas atualizações de peso dependem dos gradientes calculados durante o processo de retropropagação. Isso significa que o desempenho e o comportamento da célula LSTM são diretamente afetados pela eficácia do algoritmo de treinamento em aprender padrões nas sequências de entrada e saída. Uma configuração cuidadosa e um treinamento adequado são essenciais para obter o melhor desempenho de uma célula LSTM em uma determinada tarefa.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.3 Diferença entre GRU e LSTM

As Long Short-Term Memory (LSTMs) e as Gated Recurrent Units (GRUs) são variações das Redes Neurais Recorrentes (RNNs) projetadas para resolver um dos principais problemas enfrentados por RNNs tradicionais: a dificuldade de capturar dependências de longo prazo devido ao desvanecimento do gradiente.

Apesar de sua maior eficiência, a GRU pode ter dificuldades para lidar com sequências muito longas, pois tende a esquecer informações mais rapidamente do que a LSTM. Enquanto a LSTM é capaz de capturar dependências de longo prazo com mais eficácia, a GRU apresenta um comportamento mais dinâmico e pode ser considerada uma opção melhor quando a relação entre elementos da sequência não se estende por muitos passos temporais. Isso significa que a escolha entre essas duas arquiteturas deve considerar a complexidade da tarefa e a necessidade de retenção de informações ao longo do tempo.

De modo geral, a decisão entre usar LSTM ou GRU depende das características do problema a ser resolvido. Se a prioridade for desempenho e velocidade, a GRU pode ser a melhor opção, pois consegue capturar padrões temporais sem exigir tanto poder computacional. Por outro lado, se a tarefa exigir memória de longo prazo, como análise de textos extensos ou previsão de tendências complexas, a LSTM pode proporcionar melhores resultados. Ambas as abordagens são extremamente úteis e representam avanços importantes no aprendizado com dados sequenciais, permitindo que as RNNs lidem de maneira muito mais eficaz com padrões temporais e relações complexas ao longo do tempo.

3.4 Fluxo de informações de uma célula LSTM

O processo final de uma célula LSTM envolve a combinação de entradas e saídas por meio de portões ponderados e funções de ativação. Os portões de entrada, esquecimento e saída calculam as saídas ativadas sigmoidalmente a partir de combinações lineares das entradas e do estado oculto anterior. Essas saídas determinam quais informações serão adicionadas na memória da célula, quais serão mantidas ou esquecidas, além de quanto da memória atual será exposta como saída. As funções de ativação sigmoide permitem que esses portões controlem o fluxo de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

informações com base em valores entre 0 e 1, influenciando a magnitude das contribuições das entradas e do estado oculto anterior na memória da célula e na saída final.

O resultado esperado do processo é uma saída representativa que capture dependências de longo prazo em sequências de dados. Isso é alcançado pela capacidade dos portões de controlar cuidadosamente quais informações são atualizadas, mantidas ou esquecidas na memória da célula, permitindo que a LSTM aprenda padrões complexos e retenha informações relevantes ao longo do tempo. A combinação ponderada das entradas e do estado oculto anterior, juntamente com as funções de ativação sigmoide, garante que a saída final da célula LSTM seja uma representação útil e informativa do contexto da sequência de entrada.

3.5 Melhores práticas

Treinar uma LSTM, como toda Rede Neural Artificial (RNA), requer uma série de cuidados para evitar problemas comuns como o overfitting. Para cada item selecionado, elencamos algumas estratégias a serem exploradas na definição da arquitetura da LSTM, bem como no seu treinamento:

- **Estratégias para prevenção de overfitting:**
  - **Aumento de dados:** aplica técnicas de aumento de dados ajudando a expor o modelo a uma variedade maior de exemplos, reduzindo o risco de overfitting.

Atenção: as técnicas de aumentação devem preservar o aspecto temporal/sequencial dos dados.

  - **Early Stopping:** monitora o desempenho do modelo em um conjunto de validação e interrompe o treinamento quando o desempenho começa a piorar. Isso ajuda a encontrar o ponto ideal de generalização antes que o modelo comece a se ajustar demais aos dados de treinamento.
- **Estratégias para seleção de hiperparâmetros:**
  - **Grid Search e Random Search:** explora diferentes combinações de hiperparâmetros utilizando busca em grade ou busca aleatória. Isso

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

permite encontrar a combinação ideal que otimiza o desempenho do modelo. A contrapartida é que o tempo de convergência destas combinações pode ser muito alto.

- **Cross-Validation:** utiliza a validação cruzada para avaliar o desempenho do modelo em diferentes conjuntos de dados. Isso ajuda a garantir que o modelo seja robusto e generalizável para dados não vistos.

- **Estratégias de regularização:**
  - **Dropout recorrente:** aplica o dropout recorrente em unidades de LSTM para reduzir a coadaptação entre unidades de tempo consecutivas. Isso ajuda a evitar o overfitting e melhora a capacidade do modelo de generalizar para novos dados.
  - **Regularização de peso:** utiliza a regularização de peso, como a regularização L2, para penalizar pesos maiores e evitar a complexidade excessiva do modelo. Isso ajuda a evitar o overfitting e promove uma melhor generalização.
- **Estratégias para otimização de desempenho:**
  - **Utilização de GPUs:** aproveita o poder computacional das GPUs para acelerar o treinamento de modelos LSTM. Permitindo treinar modelos maiores em menos tempo e facilitando a experimentação com diferentes arquiteturas.
  - **Paralelização de treinamento:** explora técnicas de paralelização de treinamento para distribuir o processamento em várias GPUs ou dispositivos. Isso pode reduzir o tempo de treinamento e aumentar a eficiência do processo de treinamento.

É importante mencionar que essas técnicas não garantem melhores soluções, porém, utilizá-las é altamente recomendado devido aos resultados empíricos que sugerem potencial de ganho à performance das LSTMs. Ou seja, o sucesso de qualquer rede neural depende da sua arquitetura, dos dados, e da complexidade do problema modelado. As dicas e melhores práticas listadas indicam caminhos que podem favorecer os modelos propostos para as suas aplicações.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 PREDIÇÃO DE SÉRIES TEMPORAIS

4.1 Aplicações

As séries temporais desempenham um papel essencial em diversas áreas que envolvem dados coletados ao longo do tempo. Esses dados possuem padrões sequenciais que podem ser analisados para identificar tendências, prever comportamentos futuros e embasar tomadas de decisão estratégicas. Aplicações que utilizam RNNs, LSTMs e GRUs são especialmente úteis nesses contextos, pois conseguem capturar as relações temporais entre os dados. Entre as principais áreas de aplicação, destacam-se o mercado financeiro, a meteorologia e diversas outras análises baseadas em dados temporais.

Do ponto de vista visual, as séries temporais são representadas por intervalos contínuos de dados, construídos com plotagem de linhas, como ilustrado na figura a seguir:

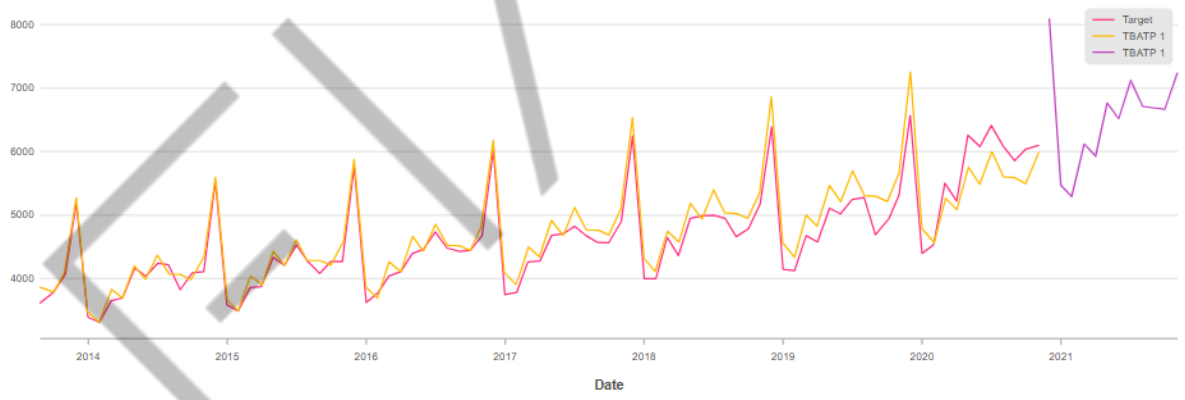


Figura 10 - Representação de séries temporais
Fonte: GitHub (2022), adaptado por FIAP (2025)

No **mercado financeiro**, a análise de séries temporais é amplamente utilizada para prever preços de ações, câmbio, commodities e outros ativos. Modelos de Aprendizado Profundo são empregados para identificar padrões de flutuação, detectar tendências e até antecipar comportamentos do mercado com base em dados históricos. Redes como LSTMs são particularmente eficazes nesse tipo de aplicação, pois conseguem capturar correlações de longo prazo entre variações nos preços e fatores externos, como indicadores macroeconômicos e eventos globais. Empresas e

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

investidores utilizam essas previsões para tomar decisões mais informadas, otimizando estratégias de compra e venda e reduzindo riscos financeiros.

Na **meteorologia**, a previsão do tempo é um dos exemplos mais tradicionais e importantes do uso de séries temporais. Dados como temperatura, umidade, velocidade do vento e pressão atmosférica são coletados continuamente e analisados para identificar padrões e antecipar mudanças climáticas. Modelos baseados em Redes Neurais Recorrentes conseguem processar grandes volumes de informações meteorológicas e gerar previsões mais precisas, auxiliando desde previsões diárias do clima até análises mais complexas, como a detecção de eventos extremos, como furacões e tempestades. Além disso, essas previsões são essenciais para setores como agricultura, aviação e gestão de desastres naturais, permitindo maior planejamento e mitigação de riscos.

Além dessas aplicações, a análise de séries temporais se estende a diversas outras áreas, como **saúde**, **transporte** e **monitoramento industrial**. No setor da saúde, por exemplo, o monitoramento de sinais vitais de pacientes, como frequência cardíaca e pressão arterial, permite a identificação precoce de anomalias e condições médicas. Em sistemas de transporte, o uso de séries temporais facilita a previsão do fluxo de tráfego, otimizando rotas e reduzindo congestionamentos. Já na indústria, sensores coletam dados continuamente para monitoramento de máquinas e previsão de falhas antes que ocorram, garantindo maior eficiência operacional e manutenção preventiva.

O estudo e a modelagem de séries temporais são essenciais para lidar com cenários dinâmicos e fazer previsões confiáveis em diversas áreas. Com os avanços em Inteligência Artificial e Aprendizado de Máquina, as aplicações dessas técnicas estão cada vez mais sofisticadas, permitindo não apenas a análise de dados históricos, mas também a tomada de decisões estratégicas em tempo real.

4.2 Características desejadas nos dados

Agora, não basta os dados serem apenas relacionados com o tempo. Se os dados estiverem desorganizados ou houver falhas na coleta, o modelo pode perder a coerência na identificação de padrões temporais. Além da **temporalidade**, uma boa

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

série temporal também precisa conter uma série de características. Vamos conhecê-las!

Outro fator importante é a **correlação temporal**, que significa que os valores passados influenciam diretamente os valores futuros. Se os dados não apresentarem essa dependência, modelos de aprendizado sequencial perdem sua eficácia, pois não há uma estrutura previsível a ser aprendida. A correlação temporal é fundamental para que a rede neural possa capturar tendências e padrões recorrentes.

A **frequência** e **regularidade** dos dados também são propriedades cruciais. Os registros devem ser coletados em intervalos constantes, pois séries temporais com espaçamentos irregulares podem dificultar a modelagem. Além disso, **dados ausentes ou com lacunas podem levar a distorções na aprendizagem, sendo necessário aplicar interpolação ou métodos de preenchimento para manter a consistência da série**.

A **tendência** e **sazonalidade** são padrões importantes que podem estar presentes nas séries temporais. A tendência ocorre quando há um crescimento ou declínio persistente nos dados ao longo do tempo. Observe:

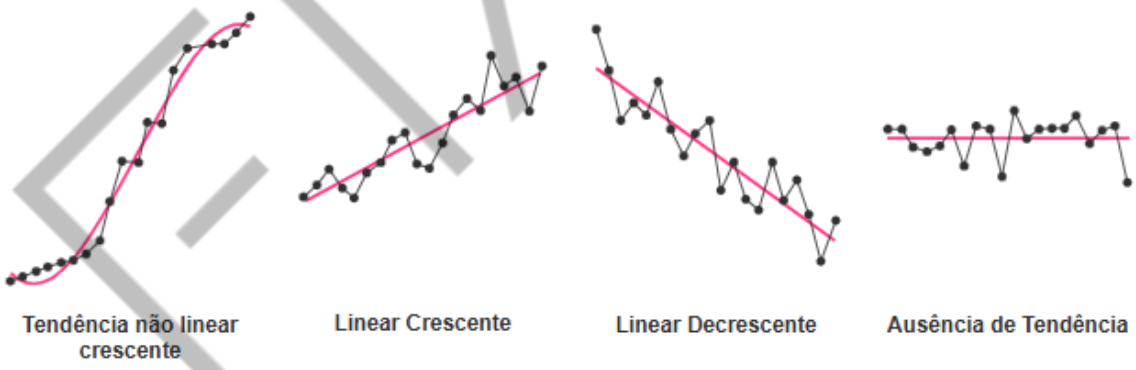


Figura 11 - Representação de tendências em séries temporais
Fonte: Medium (2024), adaptado por FIAP (2025)

Já a sazonalidade refere-se a padrões que se repetem em períodos específicos (ilustrado na figura “Representação de tendência e sazonalidade em série temporal”). A identificação dessas propriedades permite que o modelo diferencie flutuações normais de variações inesperadas, melhorando a precisão das previsões.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

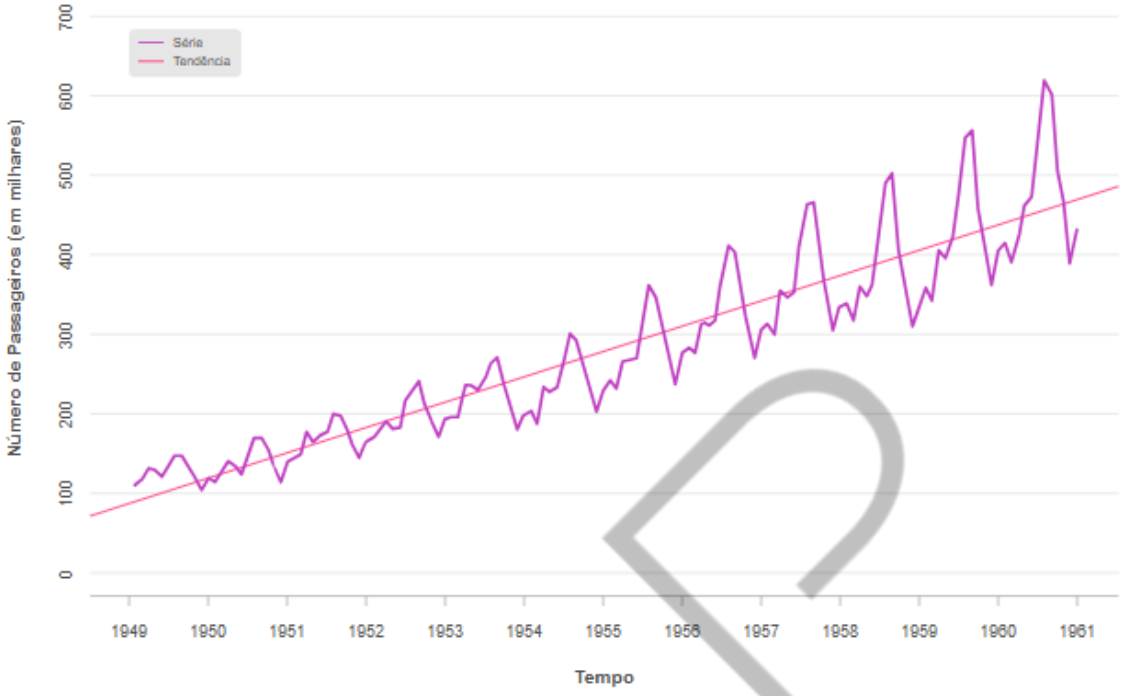


Figura 12 - Representação de tendência e sazonalidade em série temporal
Fonte: Medium (2024), adaptado por FIAP (2025)

A **estacionaridade** é uma das propriedades mais desejáveis em séries temporais, pois **indica que as estatísticas fundamentais da série**, como média, variância e autocorrelação, **permanecem constantes ao longo do tempo**. Modelos de Aprendizado de Máquina funcionam melhor quando os dados seguem um padrão estacionário, pois isso facilita a detecção de relações entre os valores.

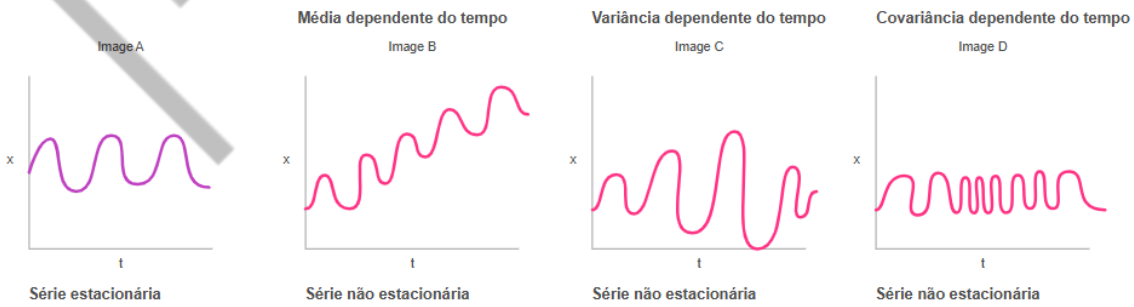


Figura 13 - Representação de tendência e sazonalidade em série temporal
Fonte: Medium (2024), adaptado por FIAP (2025)

Observação: caso uma série não seja estacionária, é comum aplicar transformações, como a diferenciação, para estabilizar a variabilidade dos dados antes da modelagem.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Por fim, a presença de **ruído** pode comprometer a qualidade da modelagem. Ruídos são variações aleatórias nos dados que não seguem um padrão lógico e podem atrapalhar a identificação das relações temporais reais. Técnicas de suavização, filtragem e remoção de outliers ajudam a minimizar esses efeitos, garantindo que a rede neural aprenda apenas os padrões significativos da série.

Portanto, para que uma série temporal seja bem modelada, é essencial que ela possua **temporalidade**, **correlação significativa**, **frequência regular**, **identificação de tendências** e **sazonalidade**, **estacionaridade** e **controle de ruído**. Quando essas características estão presentes, os modelos de aprendizado sequencial conseguem extrair informações relevantes e produzir previsões mais confiáveis e precisas.

4.3 Preparação dos dados

Antes de treinar um modelo baseado em séries temporais, é fundamental realizar uma análise exploratória detalhada da base de dados para garantir que ela possui as características desejadas. Como as séries temporais carregam informações sequenciais importantes, qualquer anomalia pode impactar diretamente o desempenho do modelo. Por isso, a verificação da temporalidade, correlação significativa, frequência regular, tendência, sazonalidade, estacionaridade e controle de ruído deve ser uma etapa obrigatória do processo de preparação dos dados.

A análise exploratória pode ser realizada através da **visualização gráfica da série, inspeção estatística das variáveis e testes formais**. Por exemplo, gráficos de séries temporais ajudam a identificar padrões sazonais e tendências, enquanto testes estatísticos, como o teste de Dickey-Fuller aumentado (ADF), verificam a estacionaridade da série. Além disso, é importante calcular autocorrelações para avaliar a influência dos valores passados na variável de interesse. Caso alguma dessas características não esteja presente de maneira adequada, é necessário transformar os dados para corrigir ou realçar esses padrões.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.3.1 Alterando a série temporal para atingir as características desejadas

Após a análise exploratória, algumas técnicas podem ser aplicadas para transformar a série original e garantir que ela possua as propriedades desejadas para a modelagem. Entre as estratégias mais comuns estão:

- **Diferenciação para garantir a estacionaridade:**
  - Se a série apresentar tendência crescente ou decrescente ao longo do tempo, a diferenciação pode ser aplicada para remover essa tendência. Isso é realizado subtraindo cada valor pelo valor anterior:

$$Y'_t = Y_t - Y_{t-1}$$

  - Essa abordagem é útil para converter séries não estacionárias em estacionárias, facilitando o aprendizado pelo modelo.
- **Sazonalidade (diferenciação sazonal):**
  - Quando um padrão sazonal é identificado, uma diferenciação sazonal pode ser aplicada, subtraindo o valor atual pelo valor da mesma época no ciclo anterior:

$$Y'_t = Y_t - Y_{t-m}$$

  - Onde ***m*** representa o período sazonal (por exemplo, 12 meses em uma série mensal).
- **Suavização para redução de ruído:**
  - Para reduzir oscilações aleatórias, métodos como **médias móveis** podem ser aplicados:

$$Y'_t = \frac{Y_{t-1} + Y_{t-2} + \cdots + Y_{t-n}}{n}$$

  - Isso ajuda a eliminar variações indesejadas sem perder a tendência subjacente.
- **Normalização e padronização para ajuste de escala:**
  - Como modelos baseados em redes neurais são sensíveis a escalas diferentes entre variáveis, a normalização pode ser utilizada para restringir

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

os valores dentro de um intervalo, como [0, 1], garantindo estabilidade no treinamento.

  - Uma alternativa é a padronização (z-score), que transforma os dados em uma distribuição com média zero e desvio padrão unitário.

- **Engenharia de Features (extração de variáveis temporais):**
  - Algumas informações temporais podem ser extraídas como variáveis adicionais, como mês do ano, dia da semana, feriados e indicadores climáticos.
  - Isso ajuda o modelo a capturar padrões ocultos que podem influenciar as previsões.

Ao aplicar essas transformações, garantimos que a série derivada resultante seja mais estável e previsível, permitindo que os modelos de Aprendizado de Máquina ou Redes Neurais Recorrentes extraiam padrões com maior eficiência e precisão. Dessa forma, a etapa de pré-processamento não apenas corrige possíveis deficiências nos dados brutos, mas também **potencializa o desempenho do modelo ao garantir que as propriedades fundamentais da série sejam respeitadas**.

4.3.2 Ajuste no formato da base de dados

A preparação de dados para modelagem de séries temporais envolve ajustar o formato da base de dados, transformar as informações para que sejam interpretáveis pelo modelo e definir estratégias adequadas para treino e teste. Como os modelos de séries temporais precisam capturar padrões dependentes do tempo, a organização dos dados e a forma como eles são divididos são diferentes de abordagens tradicionais de Aprendizado de Máquina.

A primeira etapa consiste em organizar os dados em um formato adequado para processamento. Para isso, é comum utilizar um DataFrame estruturado, onde cada linha representa uma observação em um instante de tempo, e as colunas armazenam as variáveis relevantes. A próxima figura ilustra um exemplo de tabela de dados climáticos para previsão do índice pluviométrico (chuvas):

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Data	Temperatura (ºC)	Umidade (%)	Pressão (hPa)	Velocidade do Vento (m/s)	Índice Pluviométrico (mm)
01/01/2024	28,5	75	1012	3,2	5
02/01/2024	27,8	78	1013	2,8	4,2
03/01/2024	26,1	81	1011	3,5	7,3
04/01/2024	25,6	85	1009	4	12,1
05/01/2024	26,8	82	1010	3,1	9,7

Figura 14 - Exemplo de dados de série temporal
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

Essa base contém variáveis meteorológicas medidas diariamente, sendo que o índice pluviométrico (mm) é a variável alvo a ser prevista (destacada em rosa). Para que os modelos de aprendizado consigam capturar padrões temporais, devemos transformar essa base para um formato sequencial, criando janelas deslizantes (*sliding windows*).

Por exemplo, para prever o índice pluviométrico do dia seguinte, podemos criar um formato em que cada linha contém um conjunto de observações passadas, como demonstrado na figura a seguir:

Dia Atual	Temp (t-3)	Umid (t-3)	Press (t-3)	Vento (t-3)	Índice (t-3)	Temp (t-2)	Umid (t-2)	Press (t-2)	Vento (t-2)	Índice (t-2)	Temp (t-1)	Umid (t-1)	Press (t-1)	Vento (t-1)	Índice (t-1)	Índice (t)
04/01/2024	28,5	75	1012	3,2	5	27,8	78	1013	2,8	4,2	26,1	81	1011	3,5	7,3	12,1
05/01/2024	27,8	78	1013	2,8	4,2	26,1	81	1011	3,5	7,3	25,6	85	1009	4	12,1	9,7

Figura 15 - Exemplo de dados transformado para série temporal
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

Neste caso, cada linha do novo DataFrame contém valores das últimas **três observações** para prever a chuva no dia atual (Índice(t)). Esse formato permite que as Redes Neurais Recorrentes aprendam dependências temporais entre as variáveis preditoras e a variável alvo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.3.3 Estratégias para divisão dos dados entre treino e teste

Diferente de modelos convencionais de Aprendizado de Máquina, onde os dados podem ser divididos aleatoriamente entre treino e teste, **em séries temporais é essencial manter a ordem cronológica**. A divisão deve ser realizada de maneira a refletir o fluxo do tempo real, evitando que o modelo tenha acesso a informações do futuro durante o treinamento. Duas abordagens comuns são:

1. **Divisão Treino/Teste Simples:**

- O conjunto de treino consiste nos primeiros 80% dos dados.
- O conjunto de teste corresponde aos últimos 20%, simulando previsões em novos períodos.
- Exemplo:
  - Treino: de 01-01-2024 a 30-11-2024;
  - Teste: de 01-12-2024 a 31-12-2024.

2. **Validação Expansiva (*Expanding Window Validation*):**

- A base de treino cresce progressivamente à medida que novas previsões são feitas.
- O modelo é reentreinado conforme novos dados se tornam disponíveis, simulando um sistema em tempo real.
- Exemplo:
  - Treino inicial: 01-01-2024 a 30-06-2024;
  - Teste 1: 01-07-2024 a 31-07-2024;
  - Treino atualizado: 01-01-2024 a 31-07-2024;
  - Teste 2: 01-08-2024 a 31-08-2024.
- Esse método permite que o modelo aprenda continuamente e se adapte a padrões mais recentes.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.4 Séries univariadas e multivariadas

As séries temporais podem ser classificadas como univariadas ou multivariadas, dependendo do número de variáveis envolvidas na modelagem.

Uma série temporal univariada contém apenas uma variável dependente, ou seja, o modelo faz previsões com base em uma única sequência de valores ao longo do tempo. Por exemplo, prever o índice pluviométrico (chuva) com base apenas nos valores anteriores dessa mesma variável é um caso de série univariada.

Já uma série temporal multivariada inclui múltiplas variáveis que podem influenciar a variável alvo. No caso do exemplo da chuva, onde temos temperatura, umidade, pressão atmosférica e velocidade do vento como entradas, trata-se de uma série multivariada, pois essas variáveis podem ter impacto na previsão do índice pluviométrico.

Para transformar esse conjunto de dados em uma série univariada, bastaria utilizar apenas a coluna do índice pluviométrico e prever os valores futuros baseando-se apenas nos seus valores passados. Por outro lado, para converter uma série univariada em multivariada, bastaria adicionar novas variáveis explicativas (como temperatura e umidade) ao modelo, permitindo que ele aprenda padrões mais complexos e melhore a qualidade das previsões.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 PROCESSAMENTO DE LINGUAGEM NATURAL

O Processamento de Linguagem Natural (PLN) é uma área da Inteligência Artificial que busca permitir que máquinas compreendam, interpretem e gerem linguagem humana de forma automatizada. Utilizando técnicas de Aprendizado de Máquina, Redes Neurais e Estatísticas, o PLN é aplicado em diversas tarefas, como análise de sentimentos, tradução automática, reconhecimento de fala e chatbots. O desafio dessa área está no fato de a linguagem natural ser ambígua, complexa e altamente dependente do contexto, exigindo modelos avançados, como Redes Neurais Recorrentes (RNNs, LSTMs e GRUs) e transformadores, como o BERT e o GPT, para capturar significados sutis e relações semânticas em textos e falas. Neste capítulo nosso foco é o processamento de texto.

O pipeline de tratamento de textos transforma dados não estruturados e complexos em um formato utilizável por algoritmos de Aprendizado de Máquina, permitindo a construção de modelos preditivos eficazes. Desde a limpeza inicial dos textos, passando pela representação dos dados em vetores numéricos, até a preparação final para o treinamento de modelos, cada etapa do pipeline é crucial para garantir a precisão e a relevância dos resultados.

A figura a seguir mostra que o pipeline de tratamento de texto está dividido, grosseiramente, em duas etapas: **pré-processamento** e **representação**.

Figura 16 - Etapas do pipeline de tratamento de dados
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Cada uma destas etapas se desdobra em N outras atividades, como veremos a seguir. De forma geral, dizemos que a **limpeza** elimina palavras com baixo poder informativo e padroniza sua representação (por exemplo, excluindo flexões de gênero e número). Já a **representação** codifica as “palavras resultantes da limpeza” em uma linguagem computacional, ou seja, numérica.

5.1 Limpeza

A limpeza dos textos é uma etapa essencial no pipeline de tratamento de dados textuais, pois prepara o texto bruto para ser representado de forma mais concisa, porém mantendo o significado. Cada etapa de pré-processamento tem seu objetivo específico, garantindo que o texto esteja limpo, padronizado e pronto para ser posteriormente transformado em uma representação numérica. As etapas típicas de pré-processamento são:

- **Etapa 1 – Tokenização:**
  - É o processo de dividir o texto em unidades menores, chamadas tokens, que geralmente correspondem a palavras ou frases. A tokenização é crucial porque transforma a sequência contínua de texto em elementos discretos que podem ser analisados separadamente. Esta etapa facilita a criação de representações numéricas dos dados textuais.
- **Etapa 2 – Conversão para letras minúsculas:**
  - Esta etapa envolve transformar todas as letras do texto para minúsculas. A importância dessa etapa está em garantir a consistência dos dados, já que "Palavra" e "palavra" devem ser tratadas como a mesma entidade. Isso evita a duplicação desnecessária de tokens e ajuda a simplificar a análise.
- **Etapa 3 – Remoção de quebras de linha:**
  - A remoção de quebras de linha é crucial para manter a fluidez do texto. Quebras de linha podem introduzir inconsistências e dificultar a tokenização correta. Ao remover esses caracteres, garantimos que o texto

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

permaneça em um formato contínuo, facilitando as etapas subsequentes de processamento.

- **Etapa 4 – Normalização de espaços:**
  - Normalizar espaços significa remover espaços extras e garantir que apenas um espaço separa cada palavra. Isso é importante para evitar tokens indesejados causados por múltiplos espaços e para manter a integridade das frases. Textos digitados por usuários muitas vezes contêm espaços extras que precisam ser ajustados para uma análise precisa.
- **Etapa 5 – Remoção de sinais de pontuação:**
  - A pontuação pode introduzir ruído no texto, especialmente quando o objetivo é analisar palavras ou expressões. Remover sinais de pontuação ajuda a focar no conteúdo significativo do texto, evitando que símbolos não alfabéticos interfiram na análise. No entanto, em alguns casos específicos, como a análise de sentimentos, a pontuação, emojis e emoticons podem carregar informações importantes, e essa decisão deve ser tomada com cuidado.
- **Etapa 6 – Correção ortográfica:**
  - A correção ortográfica busca ajustar erros de digitação e ortografia no texto. Isso é importante para garantir que palavras mal escritas não sejam tratadas como tokens diferentes, o que pode reduzir a eficácia do modelo. Correções ortográficas ajudam a unificar a representação das palavras e melhorar a qualidade dos dados de entrada.
- **Etapa 7 – Lematização:**
  - A lematização consiste em reduzir as palavras às suas formas base ou raiz, conhecidas como *lemmas*. Por exemplo, as palavras “correndo”, “correu” e “correrá” seriam todas reduzidas a “correr”. Isso ajuda a normalizar diferentes formas da mesma palavra, permitindo que o modelo de Aprendizado de Máquina reconheça que todas essas formas têm o mesmo significado fundamental.
- **Etapa 8 – Remoção de *stop words*:**

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- *Stop words* são palavras comuns, como "e", "o", "a", "de", "em", entre outras, que aparecem com frequência em um texto, mas geralmente não carregam muito significado semântico. Remover essas palavras pode ajudar a reduzir o ruído no texto e focar nas palavras que realmente importam para a análise. Isso é particularmente útil para melhorar a eficiência e a eficácia dos modelos de Aprendizado de Máquina.

Todas essas etapas de pré-processamento são essenciais para garantir que os dados textuais estejam limpos, consistentes e prontos para serem posteriormente representados como números. No entanto, o processo não precisa necessariamente ocorrer na ordem dada. Por exemplo, podemos remover quebras de linha e espaços antes de criar tokens. No entanto, faz sentido aplicar a correção gramatical antes de lematizar.

5.2 Representação de texto

Após o pré-processamento, a próxima etapa no pipeline de tratamento de dados textuais é a representação. **A representação envolve transformar o texto limpo e processado em um formato numérico que possa ser compreendido e manipulado por algoritmos de Aprendizado de Máquina**. Existem várias técnicas para representar textos, cada uma possui suas vantagens e desvantagens, sendo mais adequadas para diferentes tipos de tarefas de análise de texto. Veremos algumas a seguir.

A escolha da técnica de representação adequada depende da natureza do problema, do tipo de análise desejada e dos recursos computacionais disponíveis. Representar o texto de forma eficaz é fundamental para garantir que os modelos de Aprendizado de Máquina possam extrair insights valiosos e realizar predições precisas.

5.2.1 Bag of Words (BoW)

O método Bag of Words (BoW) é uma técnica fundamental e amplamente utilizada na representação de textos para Aprendizado de Máquina e NLP. Apesar de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

sua simplicidade, é uma ferramenta poderosa para transformar dados textuais em uma forma que pode ser utilizada por algoritmos de aprendizado. A ideia central do BoW é representar um texto como uma **coleção** (ou “saco”) **de suas palavras**, **desconsiderando a ordem e a gramática, mas mantendo a multiplicidade de ocorrências de cada palavra**.

Para entender como o BoW funciona, considere um conjunto de documentos ou textos. Primeiramente, é necessário construir um **vocabulário**, que é a lista de todas as palavras únicas presentes em todo o conjunto de textos. Cada palavra no vocabulário é, então, associada a um índice único. Este vocabulário serve como base para a criação dos vetores de representação de cada texto.

Uma vez que o vocabulário está definido, cada documento é representado como um vetor de frequência de palavras. Este vetor tem o mesmo tamanho do vocabulário, e cada posição no vetor corresponde à frequência com que a palavra associada aparece no documento. Por exemplo, se o vocabulário consiste nas palavras ["gato", "cachorro", "pássaro"], e o texto "gato e cachorro" é processado, o vetor de frequência seria [1, 1, 0], indicando que "gato" e "cachorro" aparecem uma vez cada, enquanto "pássaro" não aparece. Observe a figura:

Vocabulário

Textos	about	bird	heard	is	the	word	you	Representação Numérica
About the bird, the bird, bird bird bird	1	5	0	0	2	0	0	→ [1, 5, 0, 0, 2, 0, 0]
You heard about the bird	1	1	1	0	1	0	1	→ [1, 1, 1, 0, 1, 0, 1]
The bird is the word	0	1	0	1	2	1	0	→ [0, 1, 0, 1, 2, 1, 0]

Figura 17 - Etapas do pipeline de tratamento de dados
Fonte: DataCamp (2024), adaptado por FIAP (2025)

Uma característica importante do BoW é que ele **desconsidera completamente a ordem das palavras**. Isso significa que as frases "cachorro e gato" e "gato e cachorro" terão a mesma representação vetorial. Embora essa perda de informação contextual possa parecer uma desvantagem, muitas aplicações de Aprendizado de Máquina se beneficiam dessa simplificação, especialmente em problemas de classificação de texto onde a presença de certas palavras é mais importante do que sua ordem.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No entanto, o BoW tem suas limitações. Primeiramente, a dimensão dos vetores pode se tornar muito alta, especialmente com vocabulários grandes, levando ao problema de alta dimensionalidade. Além disso, como mencionado, o BoW ignora a ordem e a estrutura das palavras, o que pode ser uma desvantagem em tarefas que dependem fortemente do contexto e da sintaxe, como a análise de sentimentos em textos complexos.

Para mitigar algumas dessas limitações, outras técnicas podem ser combinadas com o BoW, como, por exemplo, o **Term Frequency-Inverse Document Frequency** (TF-IDF).

5.2.2 Term Frequency-Inverse Document Frequency (TF-IDF)

A técnica Term Frequency-Inverse Document Frequency (TF-IDF) é uma abordagem avançada para a representação de texto que visa superar algumas das limitações do método BoW. A intuição por trás do TF-IDF é que não apenas a frequência de uma palavra em um documento é importante, mas também o quão rara ou comum essa palavra é em todo o corpus de documentos. A ideia é dar mais peso a palavras que são frequentes em um documento específico, mas raras em outros documentos, pois essas palavras são mais informativas e distintivas.

O TF-IDF é composto de duas partes principais: a Frequência de Termos (*Term Frequency* — TF) e a Frequência Inversa de Documentos (*Inverse Document Frequency* — IDF).

- **Frequência de Termos (TF):** a TF é uma medida de quantas vezes uma palavra aparece em um documento em comparação ao número total de palavras nesse documento. A fórmula básica para calcular a TF de uma palavra ***t*** em um documento ***d*** é:

$$TF(t,d) = \frac{Número\ de\ vezes\ que\ a\ palavra\ \boldsymbol{t}\ aparede\ em\ \boldsymbol{d}}{Número\ total\ de\ palavras\ em\ \boldsymbol{d}}$$

- **Frequência Inversa de Documentos (IDF):** a IDF mede a raridade de uma palavra em todo o corpus. A ideia é que palavras comuns, como "e", "o", "a", devem ter menos peso, enquanto palavras raras devem ter mais peso. A fórmula básica para calcular a IDF de uma palavra ***t*** é:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

$$IDF(t) = \log\left(\frac{Número\ total\ de\ documentos}{Número\ de\ documentos\ que\ contêm\ \boldsymbol{t}}\right)$$

- **Combinação TF-IDF:** o valor final do TF-IDF é obtido multiplicando o TF pela IDF:

$$TF_IDF(t,d) = TF(t,d) * IDF(t)$$

Como todo método computacional, o TF-IDF possui vantagens e desvantagens, que estão resumidas no quadro a seguir:

Vantagens	Desvantagens
Relevância informativa: o TF-IDF destaca palavras que são mais informativas para a diferenciação entre documentos, reduzindo o impacto de palavras muito comuns que aparecem em quase todos os textos.	**Perda de contexto:** assim como o BoW, o TF-IDF ignora a ordem e a estrutura das palavras, o que pode ser uma limitação em tarefas que dependem fortemente do contexto e da sintaxe.
Redução de ruído: ao atribuir menos peso a palavras frequentes, o TF-IDF reduz o ruído e melhora a qualidade das representações textuais.	**Alta dimensionalidade:** em grandes conjuntos de dados textuais, a dimensionalidade dos vetores de TF-IDF pode ser muito alta, levando ao problema de esparsidade e exigindo mais recursos computacionais.
Simplicidade e eficiência: apesar de ser um método mais sofisticado que o BoW, o TF-IDF ainda é relativamente simples de implementar e computacionalmente eficiente.	**Atualização de pesos:** a necessidade de recalcular os valores de IDF à medida que novos documentos são adicionados ao corpus pode ser computacionalmente intensiva e complexa.

Quadro 1 - Vantagens e desvantagens do TF-IDF
Fonte: Elaborado pelo autor (2025)

Em resumo, o TF-IDF supera algumas das limitações do BoW ao ponderar a frequência das palavras com sua raridade em todo o corpus, mas ainda enfrenta desafios relacionados à perda de contexto e alta dimensionalidade. No entanto, sua simplicidade e eficácia tornam o TF-IDF uma escolha popular para muitas aplicações de Processamento de Linguagem Natural.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.2.3 Embeddings de palavras

Os embeddings de palavras são uma técnica avançada para representar palavras em um espaço vetorial contínuo, permitindo que algoritmos de Aprendizado de Máquina capturem relações semânticas e contextuais entre palavras. Ao contrário dos métodos tradicionais como Bag of Words (BoW) e TF-IDF, os embeddings de palavras criam vetores densos e de baixa dimensão que preservam as semelhanças semânticas entre palavras. Três das técnicas mais populares para gerar embeddings de palavras são Word2Vec, GloVe e FastText.

- **Word2Vec:** desenvolvido por uma equipe do Google liderada por Tomas Mikolov, o Word2Vec é uma das técnicas mais influentes e amplamente utilizadas para criar embeddings de palavras. Word2Vec utiliza uma Rede Neural de duas camadas para treinar embeddings a partir de grandes conjuntos de dados textuais. Existem dois modelos principais em Word2Vec: Continuous Bag of Words (CBOW) e Skip-gram.
    - **CBOW:** este modelo prediz a palavra alvo com base no contexto circundante. Ou seja, **dado um conjunto de palavras de contexto, o CBOW tenta prever a palavra central**. Isso ajuda a capturar a semântica da palavra em relação ao seu contexto imediato.
    - **Skip-gram:** ao contrário do CBOW, o **Skip-gram tenta prever as palavras de contexto a partir da palavra central**. Este modelo é particularmente eficaz para capturar relações semânticas entre palavras que estão distantes umas das outras no texto.

A principal vantagem do Word2Vec é sua capacidade de capturar relações semânticas complexas entre palavras, como "rei" e "rainha", "homem" e "mulher". As operações vetoriais simples, como subtrair "homem" de "rei" e adicionar "mulher", resultam em um vetor próximo ao de "rainha".

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

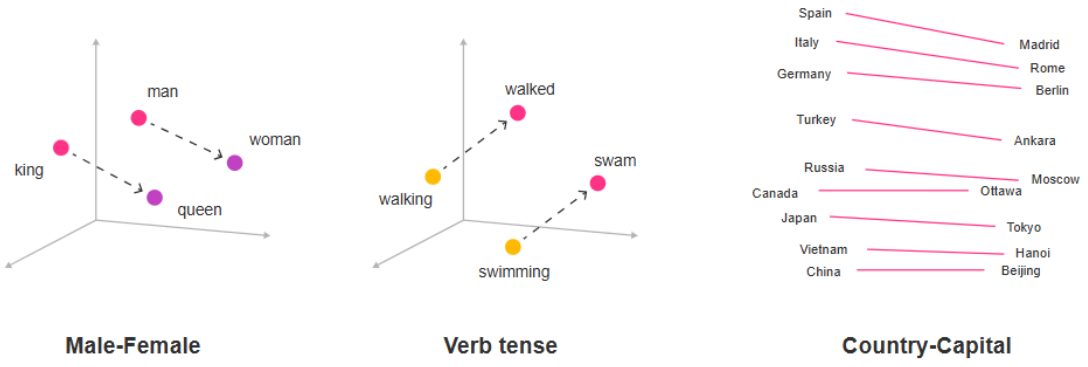


Figura 18 - Intuição do Word2Vec
Fonte: LinkedIn (2018), adaptado por FIAP (2025)

- **GloVe (Global Vectors for Word Representation):** o GloVe é outra técnica popular para criar embeddings de palavras, desenvolvida pela equipe de Stanford. Diferente do Word2Vec, que treina uma Rede Neural para prever palavras em um contexto local, o **GloVe se baseia na matriz de coocorrência global das palavras em um corpus**. A ideia central é que a relação entre as palavras pode ser capturada pela frequência com que elas aparecem juntas no texto.

  GloVe constrói uma matriz de coocorrência onde cada entrada representa a frequência com que duas palavras aparecem juntas dentro de uma janela de contexto. Em seguida, uma fatoração da matriz é realizada para obter vetores de palavras densos e de baixa dimensão que preservam as relações semânticas e contextuais.

  Uma vantagem significativa do GloVe é sua capacidade de capturar tanto as relações locais quanto as globais entre palavras, combinando a eficácia de técnicas baseadas em contagem e previsão.

- **FastText:** desenvolvido pelo Facebook AI Research (FAIR), o FastText é uma extensão do Word2Vec que leva em consideração subpalavras ou n-gramas de caracteres. Enquanto o Word2Vec e o GloVe tratam cada palavra como uma unidade atômica, **o FastText decompõe palavras em subpalavras menores durante o treinamento**. Por exemplo, a palavra "caminhando" pode ser decomposta em trigramas como "cam", "ami", "min", "inh", "ndo".

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A principal vantagem do FastText é sua **capacidade de lidar com palavras raras e palavras fora do vocabulário**. Como ele treina embeddings de subpalavras, pode gerar representações para palavras desconhecidas ao combinar embeddings de seus componentes. Isso é particularmente útil para lidar com idiomas morfologicamente ricos, onde as palavras podem ter muitas formas diferentes.

Resumindo, podemos dizer que os embeddings de palavras representam avanços significativos na representação de texto. Eles permitem que os algoritmos de Aprendizado de Máquina capturem relações semânticas e contextuais entre palavras de maneira mais eficaz do que os métodos tradicionais. De forma geral, temos:

- **Word2Vec**, conhecido por suas relações semânticas precisas;
- **GloVe** por combinar contextos locais e globais;
- **FastText** por sua robustez em lidar com palavras raras e fora do vocabulário.

Essas técnicas têm transformado a forma como os textos são processados e analisados, abrindo novas possibilidades para aplicações de NLP.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6 TRANSFORMERS *VERSUS* RNNS: O DECLÍNIO DAS LSTMS?

Nos últimos anos, os Transformers e o modelo BERT (Bidirectional Encoder Representations from Transformers) têm representado um grande avanço nas técnicas de NLP. Esses modelos têm revolucionado a forma como os textos são analisados, oferecendo capacidades incomparáveis de compreensão contextual e semântica.

6.1 Por que os transformers substituíram as RNNs em NLP?

Os transformers substituíram as RNNs no Processamento de Linguagem Natural porque oferecem maior eficiência e precisão na modelagem de dependências de longo prazo. Enquanto RNNs, LSTMs e GRUs processam sequências de forma recorrente e sequencial, tornando o treinamento mais lento e limitando a capacidade de paralelização, os transformers utilizam um mecanismo chamado *self-attention*, que permite processar todos os tokens de uma sentença simultaneamente. Isso elimina problemas como o desvanecimento do gradiente e melhora a captação de relações complexas entre palavras, independentemente da distância entre elas no texto. Além disso, modelos como BERT e GPT, baseados em transformers, conseguem aprender representações contextuais mais ricas, tornando-se o padrão para tarefas como tradução, resumo de textos e chatbots inteligentes.

6.2 Diferenças fundamentais: atenção *versus* recorrência

A principal diferença entre atenção e recorrência está na forma como processam sequências. A recorrência (usada em RNNs, LSTMs e GRUs) processa os dados sequencialmente, onde cada estado depende do anterior, tornando o treinamento mais lento e sujeito a perda de informações em sequências longas. Já a atenção (utilizada em transformers) permite que o modelo processe toda a sequência simultaneamente e foque nas partes mais relevantes do texto através do mecanismo de *self-attention*. Isso melhora a captura de dependências de longo prazo, aumenta a eficiência computacional e elimina as limitações das redes recorrentes.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6.2.1 Transformers

Introduzido no artigo “Attention is All You Need” por Vaswani *et al*. em 2017, o Transformer é uma arquitetura de Rede Neural que utiliza mecanismos de atenção para processar dados sequenciais. Diferente das Redes Neurais Recorrentes (RNNs), os Transformers não processam a sequência de entrada de maneira ordenada, palavra por palavra. Em vez disso, eles aplicam uma operação de atenção que permite que o modelo considere todas as palavras do texto de entrada ao mesmo tempo. Esse mecanismo de atenção permite que o Transformer capture dependências a longo prazo de maneira mais eficaz e eficiente.

A principal inovação dos Transformers é a **atenção auto-regressiva** (*self-attention*), que calcula pesos para cada palavra em relação a todas as outras palavras na sequência. Isso significa que, para cada palavra, o modelo pode prestar atenção às palavras mais relevantes do contexto, permitindo uma compreensão mais profunda do significado. Além disso, a arquitetura do Transformer facilita o treinamento paralelo, tornando o processo mais rápido em comparação com as RNNs.

6.2.2 BERT

Desenvolvido pela equipe do Google AI Language em 2018, o BERT é um dos modelos baseados em Transformers mais conhecidos e amplamente utilizados. A inovação chave do BERT é seu treinamento bidirecional, onde o modelo aprende a partir de todo o contexto ao redor de uma palavra (tanto à esquerda quanto à direita), ao contrário dos modelos tradicionais que olham apenas para o contexto à esquerda (ou direita) de uma palavra.

O treinamento do BERT envolve duas etapas principais: **pré-treinamento** e **fine-tuning**. No pré-treinamento, o BERT é treinado em um enorme corpus de texto (como o Wikipédia e o BooksCorpus) utilizando duas tarefas: a **Modelagem de Linguagem Mascarada** (MLM), onde algumas palavras do texto são mascaradas aleatoriamente e o modelo deve prever essas palavras, e a **Previsão de Próxima Sentença** (NSP), onde o modelo aprende a prever se uma frase segue logicamente outra. Esse pré-treinamento permite que o BERT capture uma compreensão profunda e bidirecional da linguagem.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Após o pré-treinamento, o BERT é ajustado (*fine-tuned*) para tarefas específicas, como classificação de texto, respostas a perguntas, e análise de sentimentos, utilizando um conjunto menor de dados etiquetados. A capacidade do BERT poder ser ajustado para diferentes tarefas com ajustes mínimos, tornou-o uma ferramenta poderosa e versátil para diversas aplicações de NLP.

Os modelos Transformers e BERT foram fundamentais para o desenvolvimento do GPT (Generative Pre-trained Transformer), e consequentemente do ChatGPT. Isso demonstra o poder da arquitetura Transformer e o valor do pré-treinamento em grandes volumes de dados textuais, abrindo novas possibilidades para a Inteligência Artificial. Mas isso é assunto para outro momento!

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

7 HANDS ON: APLICAÇÕES DE SÉRIES TEMPORAIS

Agora que conhecemos as LSTMs, está na hora de praticarmos sua implementação. Toda nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab), usando o TensorFlow.

Nós faremos três códigos diferentes: um para série temporal univariada, outro para série temporal multivariada e o terceiro para classificação de e-mails.

Dica: veja o material HOW TO para acessar o ambiente do Colab.

7.1 Predição de ações (série univariada)

Uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 4a), e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

Atenção: para este código do capítulo, utilize os arquivos de apoio "**LSTM_TATA_train.csv**" e "**LSTM_TATA_test.csv**".

Sobre a base de dados:

A **Tata Steel Stock Dataset** (2015-2021), disponível no Kaggle, é uma coleção de dados que contém informações históricas sobre os preços das ações da Tata Steel, uma empresa multinacional de fabricação de aço. As variáveis incluídas no conjunto de dados geralmente abrangem o preço de abertura, fechamento, alta, baixa, volume de negociação e possíveis indicadores técnicos. Esses dados são valiosos para a previsão de preços de ações utilizando modelos de Deep Learning, especialmente as RNNs, devido à natureza sequencial e dependente no tempo das séries temporais financeiras. Com base em padrões históricos, as RNNs podem capturar relações temporais complexas e fornecer previsões precisas sobre os preços futuros das ações da Tata Steel, auxiliando investidores e analistas na tomada de decisões informadas.

Vamos para o código!

Começaremos com os *imports*:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Imports
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler

from keras.models import Sequential
from keras.layers import LSTM, Dropout, Dense

from tensorflow.keras import layers, models
```

Código-fonte 1 - Imports do código de LSTM
Fonte: Elaborado pelo autor (2025)

Este primeiro código não gera saída. Vamos carregar o dataset:

```
# Leitura do dataset
df = pd.read_csv("LSTM_TATA_train.csv")
df
```

Código-fonte 2 - Carregando o dataset
Fonte: Elaborado pelo autor (2025)

	Date	Open	High	Low	Last	Close	Total Trade Quantity	Turnover (Lacs)
0	2018-09-28	234.05	235.95	230.20	233.50	233.75	3069914	7162.35
1	2018-09-27	234.55	236.80	231.10	233.80	233.25	5082859	11859.95
2	2018-09-26	240.00	240.00	232.50	235.00	234.25	2240909	5248.60
3	2018-09-25	233.30	236.75	232.00	236.25	236.10	2349368	5503.90
4	2018-09-24	233.55	239.20	230.75	234.00	233.30	3423509	7999.55
...	...	...	...	...	...	...	...	...
2030	2010-07-27	117.60	119.50	112.00	118.80	118.65	586100	694.98
2031	2010-07-26	120.10	121.00	117.10	117.10	117.60	658440	780.01
2032	2010-07-23	121.80	121.95	120.25	120.35	120.65	281312	340.31
2033	2010-07-22	120.30	122.00	120.25	120.75	120.90	293312	355.17
2034	2010-07-21	122.10	123.00	121.05	121.10	121.55	658666	803.56

2035 rows × 8 columns

Figura 19 - Saída do Código-fonte 2
Fonte: Elaborado pelo autor (2025)

A figura anterior mostra que o arquivo de treino possui 2035 linhas com 8 colunas, cada uma representando uma variável. Entre elas, a mais importante é a data (Date), indicando a temperabilidade daquela informação.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Nossa análise será predizer valores futuros da ação da empresa com base nos valores passados. Para isso podemos utilizar qualquer coluna relativa ao valor da ação, desde que sigamos sempre com a mesma escolha. Por exemplo: podemos utilizar a coluna “Close” para analisar o histórico do valor da ação na abertura da bolsa e tentar prever os valores futuros da ação na abertura da bolsa. Não podemos misturar as colunas pois a tendência perderia o sentido.

Sem perda de generalidade, vamos adotar como escolha do valor da ação seu preço na abertura da bolsa (Open). Nós sabemos que os dados já estão em ordem temporal, então, o nosso “dataset de treino” será os valores desta coluna. Assim, vamos selecionar o array Numpy que com estes valores por meio do seguinte código:

```
# Puxando apenas a coluna do valor da ação na ABERTURA
training_set = df.iloc[:,1:2].values

type(training_set)
```

Código-fonte 3 - Selecionando apenas a coluna “Open”
Fonte: Elaborado pelo autor (2024)

```
numpy.ndarray
```

Figura 20 - Saída do Código-fonte 3
Fonte: Elaborado pelo autor (2025)

A figura “Saída do Código-fonte 3” mostra que nossa seleção consiste apenas de um vetor de informações, não mais uma tabela (pois utilizamos apenas uma variável). Podemos analisar as tendências históricas das ações da Tata na abertura da bolsa por meio de um gráfico, gerado com o seguinte código:

```
# Plotando a série histórica
plt.figure(figsize=(20, 9))
plt.plot(range(df.shape[0]),
         df["Open"][-1::-1])
plt.title("Série temporal da ação")
plt.xticks(range(0, df["Date"].shape[0], 50),
           df["Date"][-1:0:-50],
           rotation=45)
plt.xlabel("Datas")
plt.ylabel("Preço de abertura")
plt.show()
```

Código-fonte 4 - Plotando a série histórica dos dados
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Note que o código faz algumas “mágicas” para produzir um gráfico que faça sentido para nós, humanos. Os dados no arquivo estão do mais recente para o mais novo, mas o gráfico deve trazer as informações da mais antiga para a mais recente. Assim, a seleção dos dados ocorre em ordem inversa, da última para a primeira, pulando de 1 em 1 (o que justifica o *slicing* “`[-1::-1]`”).

A mesma lógica se aplica para o eixo X, das datas, mas para evitar um eixo muito poluído, plotamos os valores das datas apenas de 50 em 50 números, com uma rotação de 45º. O resultado está na próxima figura:

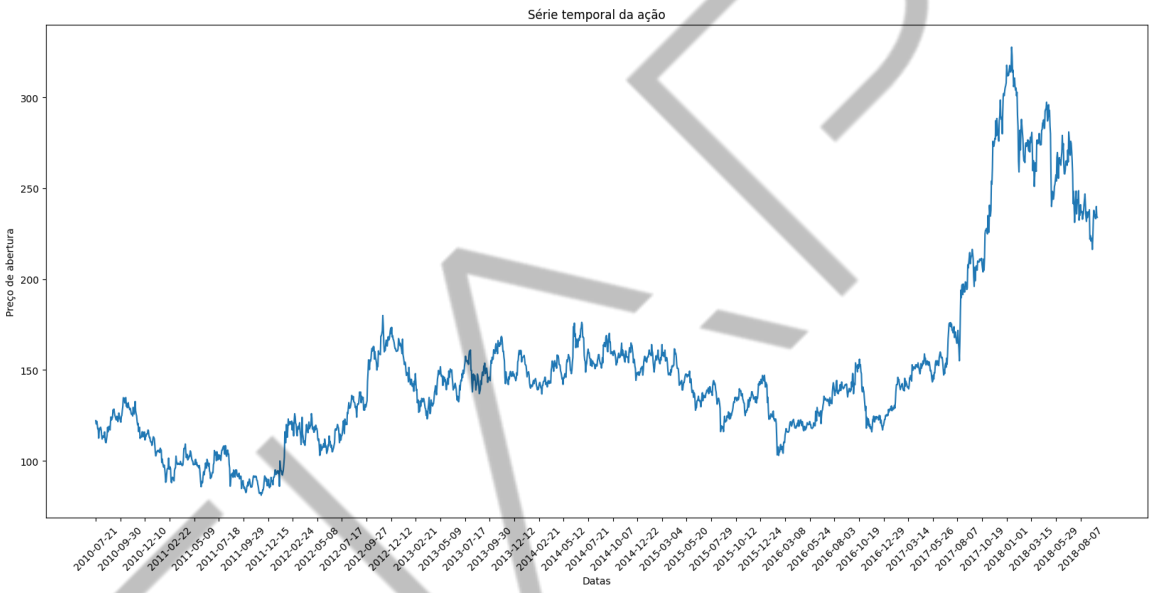


Figura 21 - Saída do Código-fonte 4
Fonte: Elaborado pelo autor (2025)

Podemos ver que em outubro de 2017 houve um estouro do preço da ação, com consequentes quedas (apesar das oscilações) até agosto de 2018.

Como estamos trabalhando com apenas 1 variável, ela claramente já está “escalada”, mas sabemos que as Redes Neurais performam melhor quando os dados estão expressos entre 0 e 1. Portanto, vamos padronizar os dados por meio do seguinte comando:

```
# Padronizando os dados
mm = MinMaxScaler()
training_set_scaled = mm.fit_transform(training_set)

training_set_scaled
```

Código-fonte 5 - Padronizando a escala dos dados entre 0 e 1
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
array([[0.6202352 ],
       [0.62226277],
       [0.64436334],
       ...,
       [0.16504461],
       [0.15896188],
       [0.16626115]])
```

Figura 22 - Saída do Código-fonte 5
Fonte: Elaborado pelo autor (2025)

Agora estamos prontos para criar nosso dataset temporal! Vimos que as séries temporais não são expressas apenas por 1 valor, mas sim por uma sequência de valores. Então, **cada exemplo do dataset de treinamento (e teste) precisa conter uma série de valores sequenciais**. O label é o valor seguinte. Construímos o dataset por meio de uma janela deslizante sobre os dados.

Imagine que a janela tenha tamanho 50. Os primeiros 50 números compõem um exemplo para a rede aprender, sendo que seu label será o 51º valor. O próximo exemplo vai do 1º valor até o 51º, sendo que seu label é o 52º valor. O próximo exemplo vai do 2º valor até o 52º, sendo que seu label é o 53º valor. E assim sucessivamente.

Vamos construir os exemplos e labels das ações utilizando uma janela deslizante de **tamanho 60**, percorrendo todos os dados disponíveis no arquivo “train”. O código a seguir não gera saída.

```
# Criando o dataset da série temporal
X_train = []
y_train = []
for i in range(60, 2035):
  X_train.append( training_set_scaled[i-60:i, 0] )
  y_train.append( training_set_scaled[i, 0] )

X_train, y_train = np.array(X_train), np.array(y_train)
```

Código-fonte 6 - Lógica para criar o dataset da série temporal
Fonte: Elaborado pelo autor (2025)

Como nosso dado possui apenas uma dimensão (isto é, apenas uma variável), vamos reescalar a variável “`X_train`” com o código:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Reescalando o formato do treino
X_train = np.reshape(X_train, (X_train.shape[0],
                               X_train.shape[1],
                               1))

X_train.shape
```

Código-fonte 7 - Reescalando o formato da variável de treino
Fonte: Elaborado pelo autor (2025)

O código-fonte anterior gera a saída da figura “Saída do Código-fonte 7”, a qual aponta que agora a variável “`X_train`” possui três dimensões, sendo 1975 linhas (exemplos de treinamento), 60 colunas (cada exemplo possui 60 valores sequenciais) e 1 série de dados (estamos lidando com apenas 1 variável).

```
(1975, 60, 1)
```

Figura 23 - Saída do Código-fonte 7
Fonte: Elaborado pelo autor (2025)

Agora estamos prontos para definir, compilar e treinar a Rede Neural Recorrente. Faremos isso por meio do seguinte código:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Definindo a arquitetura da rede
reg = Sequential()

reg.add( LSTM(units=50,
              return_sequences=True,
              input_shape=(X_train.shape[1], 1)) )
reg.add( Dropout(0.2) )

reg.add( LSTM(units=50,
              return_sequences=True) )
reg.add( Dropout(0.2) )

reg.add( LSTM(units=50,
              return_sequences=True) )
reg.add( Dropout(0.2) )

reg.add( LSTM(units=50) )
reg.add( Dropout(0.2) )

reg.add( Dense(units=1) )

# Compilando a rede montada
reg.compile(optimizer="adam", loss="mean_squared_error")

# Treinando a rede
with tf.device('/device:GPU:0'):
  reg.fit(X_train,
          y_train,
          epochs=100,
          batch_size=32)
```

Código-fonte 8 - Definindo, compilando e treinando a LSTM
Fonte: Elaborado pelo autor (2025)

A arquitetura definida é uma LSTM composta por quatro camadas, cada uma com 50 unidades de memória, seguida por camadas de dropout para regularização. A estrutura segue o padrão "`return_sequences=True`", isso significa que cada camada LSTM retorna sequências para a próxima camada, mantendo a informação temporalmente. A camada final é uma camada densa com uma única unidade, que é usada para prever a saída desejada. O dropout é utilizado para mitigar o overfitting, desativando aleatoriamente unidades durante o treinamento para evitar a co-adaptação.

Na compilação da rede, o otimizador Adam é escolhido, que é uma escolha comum para problemas de regressão devido à sua eficácia geral. A função de perda

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

escolhida é a MSE, que é apropriada para problemas de regressão, onde o objetivo é minimizar a diferença quadrada entre as previsões do modelo e os valores reais.

A rede é treinada por 100 épocas com um batch de tamanho 32. A GPU acelera o treinamento, aproveitando a capacidade de processamento paralelo para aumentar a eficiência computacional. Durante o treinamento, a rede é ajustada aos dados de treinamento (X_train e y_train) para aprender os padrões presentes nos dados e fazer previsões precisas.

O código-fonte “Definindo, compilando e treinando a LSTM” gera a saída parcialmente ilustrada na figura a seguir. Para acelerar o treinamento, não deixe de aplicar a GPU no treinamento da rede. Sua saída pode não ser idêntica, mas certamente será parecida.

```
Epoch 90/100
62/62 [==============================] - 1s 16ms/step - loss: 6.8978e-04
Epoch 91/100
62/62 [==============================] - 1s 20ms/step - loss: 6.5753e-04
Epoch 92/100
62/62 [==============================] - 1s 22ms/step - loss: 6.3354e-04
Epoch 93/100
62/62 [==============================] - 1s 22ms/step - loss: 6.2726e-04
Epoch 94/100
62/62 [==============================] - 1s 16ms/step - loss: 6.3746e-04
Epoch 95/100
62/62 [==============================] - 1s 16ms/step - loss: 6.2945e-04
Epoch 96/100
62/62 [==============================] - 1s 16ms/step - loss: 6.5931e-04
Epoch 97/100
62/62 [==============================] - 1s 16ms/step - loss: 7.2689e-04
Epoch 98/100
62/62 [==============================] - 1s 17ms/step - loss: 6.3233e-04
Epoch 99/100
62/62 [==============================] - 1s 16ms/step - loss: 7.2915e-04
Epoch 100/100
62/62 [==============================] - 1s 15ms/step - loss: 0.0010
```

Figura 24 - Saída do Código-fonte 8
Fonte: Elaborado pelo autor (2025)

Assim que o modelo estiver treinado, devemos avaliar seu desempenho. Para isso usaremos os dados do arquivo de teste. Ele contém dados de outros meses, isto é, além do mês de agosto de 2018.

Além da leitura do arquivo “teste”, precisamos concatená-los aos dados de treinamento para garantir que a sequência temporal seja mantida. Os dados de teste, de fato, serão os últimos da concatenação, pulando o tamanho do arquivo de teste para trás, além de também pular 60 posições anteriores (pois nossa janela deslizante é de tamanho 60).

Depois, precisamos preparar o conjunto de teste da mesma forma que preparamos o conjunto de treino.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     # Lendo os dados de teste
     df_test = pd.read_csv("LSTM_TATA_test.csv")
     real_stock_price = df_test.iloc[:, 1:2].values

     # Considerando que os dados são contínuos, temos que
concatenar os arquivos
     dataset_total = pd.concat((df["Open"],
                               df_test["Open"]),
                               axis=0)

     # Dos dados concatenados, os exemplos de teste estão no
final                                                        da
# concatenação pulando os últimos 60, devido ao tamanho da
nossa
     # janela. Aplique a padronização também (MinMax)
     inputs  =  dataset_total[len(dataset_total)-len(df_test)-
60:].values
     inputs = inputs.reshape(-1, 1)
     inputs = mm.transform(inputs)

     # Preparando os dados de teste nos mesmos moldes dos dados
de treinamento
     X_test = []
     for i in range(60, 76):
       X_test.append( inputs[i-60:i, 0] )

     # Ajustando o tamanho do teste, nos mesmos moldes do
treinamento
     X_test = np.array(X_test)
     X_test = np.reshape(X_test,
                         (X_test.shape[0], X_test.shape[1],
1))
```

Código-fonte 9 - Lendo e preparando os dados de teste
Fonte: Elaborado pelo autor (2025)

O código anterior não produz saídas, então podemos ir para o próximo passo, que é gerar as predições (que estarão expressas entre 0 e 1) para, assim, revertê-las para a escala original com a transformação inversa do MinMaxScaler():

```
     # Predizendo valores das ações
     predicted_stock_prices = reg.predict(X_test)

     # Voltando os valores preditos na escala [0, 1] para a
escala original
     predicted_stock_prices                                  =
mm.inverse_transform(predicted_stock_prices)
```

Código-fonte 10 - Predizendo ações a partir da série temporal
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A saída é bem direta, e mostra apenas a completude da predição.

```
1/1 [==============================] - 1s 1s/step
```

Figura 25 - Saída do Código-fonte 10
Fonte: Elaborado pelo autor (2025)

Para avaliarmos de fato o poder de predição do nosso modelo, vamos construir um gráfico que compara duas séries temporais: uma com os dados reais das ações da Tata ao longo do tempo e outra com os valores preditos das ações pela nossa LSTM:

```
# Mostrando os resultados
plt.plot(real_stock_price,
         color="black",
         label="Ações da TATA")
plt.plot(predicted_stock_prices,
         color="green",
         label="Ações PREDITAS da TATA")
plt.title("Predição de ações da TATA Consultoria")
plt.xlabel("Tempo")
plt.xticks(range(0, df_test["Date"].shape[0]),
           df_test["Date"].loc[-1:0:-1],
           rotation=45)
plt.ylabel("Preço da ação")
plt.legend()
plt.show()
```

Código-fonte 11 - Predizendo ações a partir da série temporal
Fonte: Elaborado pelo autor (2025)

A figura “Saída do Código-fonte 11” compara as duas séries temporais. Podemos notar que o poder preditivo do modelo começa “ruim” e vai melhorando ao decorrer do tempo, seguindo uma curva que se assemelha à tendência real:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

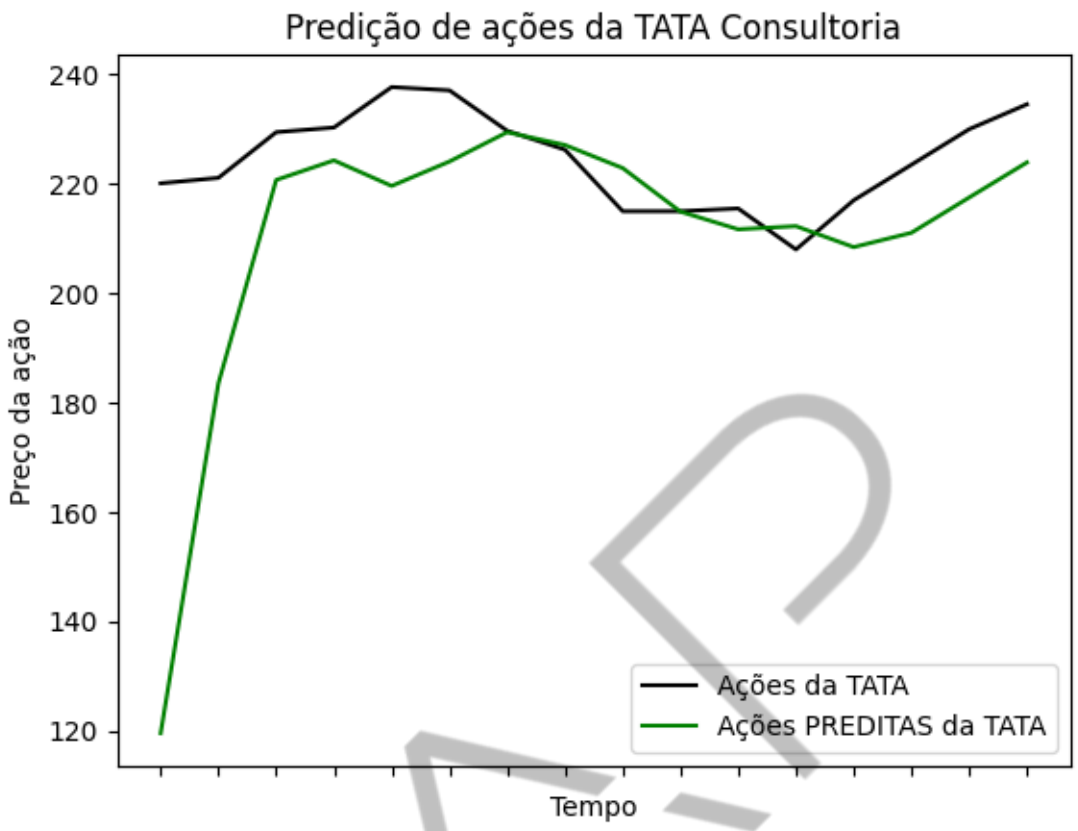


Figura 26 - Saída do Código-fonte 11
Fonte: Elaborado pelo autor (2025)

O último gráfico mostra que nosso modelo baseado em LSTM foi capaz de captar as tendências de variações do preço das ações da Tata. Ainda que não seja perfeito, ele é capaz de definir os momentos de queda e subida.

Importante: este é um modelo simples que prevê o valor futuro de ações baseado apenas na série histórica das ações da mesma empresa. Na prática, a predição de valores financeiros é influenciada por uma série de fatores externos relativos ao mercado e à economia.

7.2 Predição de poluição do ar (série multivariada)

Já no ambiente do Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 4b), e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

Dica: para este código do capítulo, vamos baixar o arquivo diretamente do repositório. Se preferir, utilize o arquivo “**raw.csv**” fornecido.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Sobre a base de dados:

Este é um conjunto de dados que reporta sobre o clima e o nível de poluição a cada hora durante cinco anos na embaixada dos EUA em Pequim, China. Os dados incluem a data e hora, a concentração de poluição chamada PM2.5, e informações meteorológicas como ponto de orvalho (DEWP), temperatura (TEMP), pressão (PRES), direção do vento (cbwd), velocidade do vento acumulada (Iws), e o número acumulado de horas de neve (Is) e chuva (Ir). A lista completa de variáveis nos dados brutos é a seguinte:

- **No:** número da linha.
- **year:** ano dos dados nesta linha.
- **month:** mês dos dados nesta linha.
- **day:** dia dos dados nesta linha.
- **hour:** hora dos dados nesta linha.
- **pm2.5:** concentração de PM2.5.
- **DEWP:** ponto de orvalho.
- **TEMP:** temperatura.
- **PRES:** pressão.
- **cbwd:** direção do vento combinada.
- **Iws:** velocidade do vento acumulada.
- **Is:** horas acumuladas de neve.
- **Ir:** horas acumuladas de chuva.

Podemos usar esses dados para formular um problema de previsão, onde, dados as condições meteorológicas e os níveis de poluição das horas anteriores, fazemos a previsão da poluição para a próxima hora.

Vamos para o código!

Começaremos baixando os dados:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
!wget
https://raw.githubusercontent.com/jbrownlee/Datasets/master/p
ollution.csv
!mv ./pollution.csv ./raw.csv
```

Código-fonte 12 - Baixando os dados brutos da predição de poluição
Fonte: Elaborado pelo autor (2025)

Agora vamos adaptar o arquivo de entrada mantendo apenas algumas variáveis mais pertinentes ao nosso problema, facilitando a manipulação via código:

```
from pandas import read_csv
from datetime import datetime

# Ajusta o formato da data
def parse(x):
  return datetime.strptime(x, '%Y %m %d %H')

# Carrega o dataset
dataset = read_csv('raw.csv', parse_dates = [['year',
'month', 'day', 'hour']], index_col=0, date_parser=parse)
dataset.drop('No', axis=1, inplace=True)

# Atribui nome às colunas
dataset.columns = ['pollution', 'dew', 'temp', 'press',
'wnd_dir', 'wnd_spd', 'snow', 'rain']
dataset.index.name = 'date'

# Altera os NaNs para zero
dataset['pollution'].fillna(0, inplace=True)

# Remove as primeiras 24 horas
dataset = dataset[24:]

# Salva o arquivo
dataset.to_csv('pollution.csv')
```

Código-fonte 13 - Adaptando os dados da série temporal
Fonte: Elaborado pelo autor (2025)

Vamos inspecionar as primeiras 10 linhas da tabela resultante (figura “Saída do Código-fonte 14”):

```
# Mostra as primeiras 10 linhas
dataset.head(10)
```

Código-fonte 14 - Mostrando as primeiras linhas
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	pollution	dew	temp	press	wnd_dir	wnd_spd	snow	rain
date								
2010-01-02 00:00:00	129.0	-16	-4.0	1020.0	SE	1.79	0	0
2010-01-02 01:00:00	148.0	-15	-4.0	1020.0	SE	2.68	0	0
2010-01-02 02:00:00	159.0	-11	-5.0	1021.0	SE	3.57	0	0
2010-01-02 03:00:00	181.0	-7	-5.0	1022.0	SE	5.36	1	0
2010-01-02 04:00:00	138.0	-7	-5.0	1022.0	SE	6.25	2	0
2010-01-02 05:00:00	109.0	-7	-6.0	1022.0	SE	7.14	3	0
2010-01-02 06:00:00	105.0	-7	-6.0	1023.0	SE	8.93	4	0
2010-01-02 07:00:00	124.0	-7	-5.0	1024.0	SE	10.72	0	0
2010-01-02 08:00:00	120.0	-8	-6.0	1024.0	SE	12.51	0	0
2010-01-02 09:00:00	132.0	-7	-5.0	1025.0	SE	14.30	0	0

Figura 27 - Saída do Código-fonte 14
Fonte: Elaborado pelo autor (2025)

Vamos explorar algumas variáveis (figura “Saída do Código-fonte 15”):

```
from pandas import read_csv
from matplotlib import pyplot

# Carrega o dataset
dataset = read_csv('pollution.csv', header=0,
index_col=0)
values = dataset.values

# Especifica as colunas para plotar
groups = [0, 1, 2, 3, 5, 6, 7]
i = 1
pyplot.figure()
for group in groups:
  pyplot.subplot(len(groups), 1, i)
  pyplot.plot(values[:, group])
      pyplot.title(dataset.columns[group], y=0.5,
loc='right')
  i += 1
pyplot.show()
```

Código-fonte 15 - Explorando algumas variáveis
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

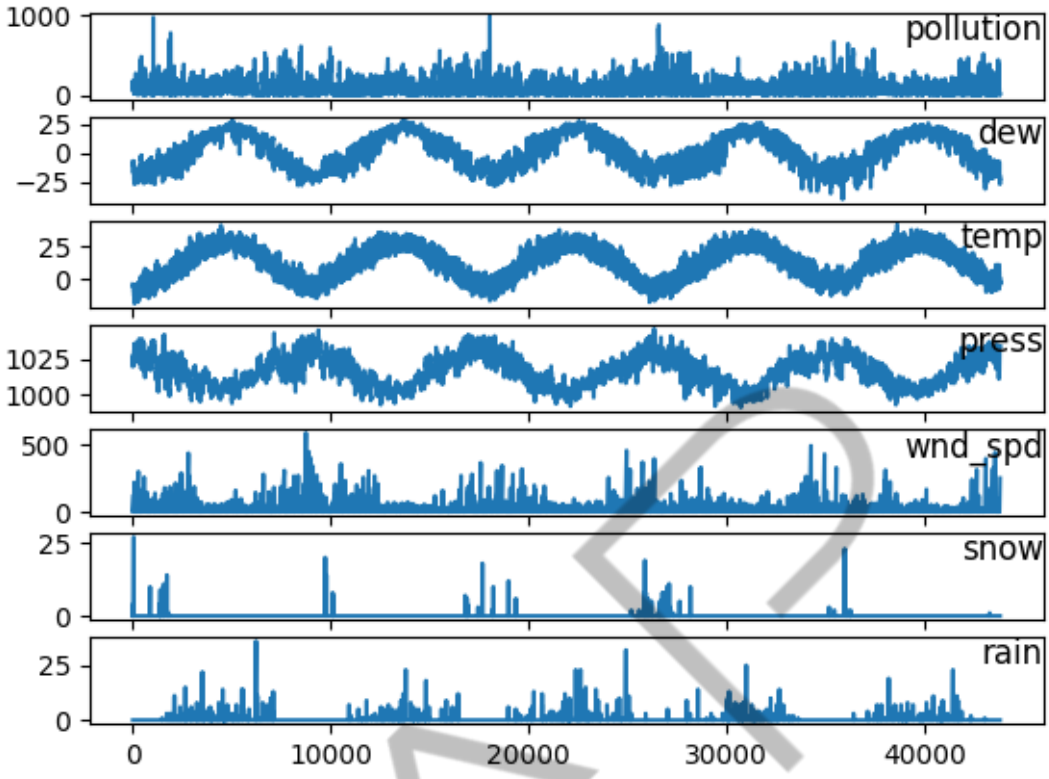


Figura 28 - Saída do Código-fonte 15
Fonte: Elaborado pelo autor (2025)

É possível notar pela figura anterior que nosso label (poluição) não está necessariamente seguindo as características desejadas de uma série temporal! Será que isso influenciará o resultado? Vamos ver!

A próxima célula contém todo o código para preparação do dataset. Destacaremos alguns pontos relevantes nas próximas imagens.

```
# Imports necessários
from math import sqrt
from numpy import concatenate
from matplotlib import pyplot
from pandas import read_csv
from pandas import DataFrame
from pandas import concat
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import mean_squared_error
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LSTM

# Converte a série em aprendizado supervisionado
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
def series_to_supervised(data, n_in=1, n_out=1, dropnan=True):
    n_vars = 1 if type(data) is list else data.shape[1]
    df = DataFrame(data)
    cols, names = list(), list()
    # input sequence (t-n, ... t-1)
    for i in range(n_in, 0, -1):
      cols.append(df.shift(i))
        names += [('var%d(t-%d)' % (j+1, i)) for j in range(n_vars)]
    # forecast sequence (t, t+1, ... t+n)
    for i in range(0, n_out):
      cols.append(df.shift(-i))
      if i == 0:
            names += [('var%d(t)' % (j+1)) for j in range(n_vars)]
      else:
          names += [('var%d(t+%d)' % (j+1, i)) for j in range(n_vars)]
    # put it all together
    agg = concat(cols, axis=1)
    agg.columns = names
    # drop rows with NaN values
    if dropnan:
      agg.dropna(inplace=True)
    return agg


# Carrega a base de dados e exclui a coluna de direção do vento (categórica)
dataset = read_csv('pollution.csv', header=0, index_col=0)
dataset.drop(columns=['wnd_dir'], inplace=True)

# Garanta que todos os dados são numéricos
values = dataset.values
values = values.astype('float32')

# Normaliza as features
scaler = MinMaxScaler(feature_range=(0, 1))
scaled = scaler.fit_transform(values)
dataset_scaled = pd.DataFrame(scaled, columns=dataset.columns)

# Escolhe a variável a ser prevista
target_variable = 'pollution'

# Separa X (features) e y (target)
X = dataset_scaled.drop(columns=[target_variable]) # Remove a variável alvo das entradas
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     y = dataset_scaled[[target_variable]]                    #
Mantém a variável alvo separada

     # Aplica a função para transformar os dados, usando 3
valores (lags) como entrada
     n_lags = 3
     df_X_supervised = series_to_supervised(X, n_in=n_lags,
n_out=0)  # Somente entradas (X)
     df_y_supervised = series_to_supervised(y, n_in=n_lags,
n_out=1)  # Apenas saída (y)

     # Juntando entradas (X) e saída (y)
     df_final = pd.concat([df_X_supervised,
df_y_supervised.iloc[:, -1]], axis=1)

     # Exibindo o DataFrame final com X e y prontos para
treinamento
     display(df_final.head())

     # Dividindo os dados em treino e teste
     values = df_final.values
     n_train_hours = 365 * 24
     train = values[:n_train_hours, :]
     test = values[n_train_hours:, :]

     # Separando features e labels
     train_X, train_y = train[:, :-1], train[:, -1]
     test_X, test_y = test[:, :-1], test[:, -1]

     # Reshape num valor 3D [samples, timesteps, features]
     train_X = train_X.reshape((train_X.shape[0],
                                n_lags,
                               int(train_X.shape[1]/n_lags)))
     test_X = test_X.reshape((test_X.shape[0],
                              n_lags,
                              int(test_X.shape[1]/n_lags)))
     print()
     display(train_X.shape, train_y.shape, test_X.shape,
test_y.shape)
```

Código-fonte 16 - Lendo e preparando os dados para a análise multivariada
Fonte: Elaborado pelo autor (2025)

Por exemplo, a figura a seguir mostra parcialmente o DataFrame gerado pela construção da série temporal com ***lag=3*** (três linhas de dados viram uma amostra para a LSTM).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	var1(t-3)	var2(t-3)	var3(t-3)	var4(t-3)	var5(t-3)	var6(t-3)	var1(t-2)	var2(t-2)	var3(t-2)	var4(t-2)	var5(t-2)	var6(t-2)	var1(t-1)	var2(t-1)	var3(t-1)	var
3	0.352941	0.245902	0.527273	0.002290	0.000000	0.0	0.367647	0.245902	0.527273	0.003811	0.000000	0.0	0.426471	0.229508	0.545454	0.0
4	0.367647	0.245902	0.527273	0.003811	0.000000	0.0	0.426471	0.229508	0.545454	0.005332	0.000000	0.0	0.485294	0.229508	0.563637	0.0
5	0.426471	0.229508	0.545454	0.005332	0.000000	0.0	0.485294	0.229508	0.563637	0.008391	0.037037	0.0	0.485294	0.229508	0.563637	0.0
6	0.485294	0.229508	0.563637	0.008391	0.037037	0.0	0.485294	0.229508	0.563637	0.009912	0.074074	0.0	0.485294	0.213115	0.563637	0.0
7	0.485294	0.229508	0.563637	0.009912	0.074074	0.0	0.485294	0.213115	0.563637	0.011433	0.111111	0.0	0.485294	0.213115	0.581818	0.0

Figura 29 - Saída do Código-fonte 16
Fonte: Elaborado pelo autor (2025)

Os tamanhos dos conjuntos de treino e teste ficaram:

- Treino: (8760, 3, 6).
- Label do treino: (8760,).
- Teste: (35037, 3, 6).
- Label do teste: (35037,).

Naturalmente, a quantidade de labels acompanha o número de amostras de cada subconjunto. Neste caso, o conjunto de treino é menor porque limitamos o treinamento a apenas os dados de 1 ano. Todo o resto ficou para o teste.

A próxima célula contém todo o código para treinamento e validação do modelo preditivo. Destacaremos alguns pontos relevantes nas próximas imagens.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     # Projeta a rede neural
     model = Sequential()
     model.add(LSTM(50,         input_shape=(train_X.shape[1],
train_X.shape[2])))
     model.add(Dense(1))
     model.compile(loss='mae', optimizer='adam')

     # Faz o fiting do modelo no treino
     history = model.fit(train_X, train_y, epochs=50,
                                  batch_size=72,  verbose=2,
shuffle=False)

     # Plota a loss do treino
     pyplot.plot(history.history['loss'], label='train')
     pyplot.legend()
     pyplot.show()

     # Faz predições no teste
     yhat = model.predict(test_X)

     # Calcula RMSE
     rmse = sqrt(mean_squared_error(test_y, yhat))
     display('Teste RMSE: %.3f' % rmse)

     # Plota as curvas do TEST_Y real e predito
     pyplot.plot(yhat[:100], label='y predito')  # Primeiros
100 pontos
     pyplot.plot(test_y[:100], label='y real')   # Primeiros
100 pontos
     pyplot.legend()
     display(pyplot.show())
```

Código-fonte 17 - Treinando e avaliando o modelo preditivo
Fonte: Elaborado pelo autor (2025)

Por exemplo, a próxima figura mostra a loss do treinamento:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

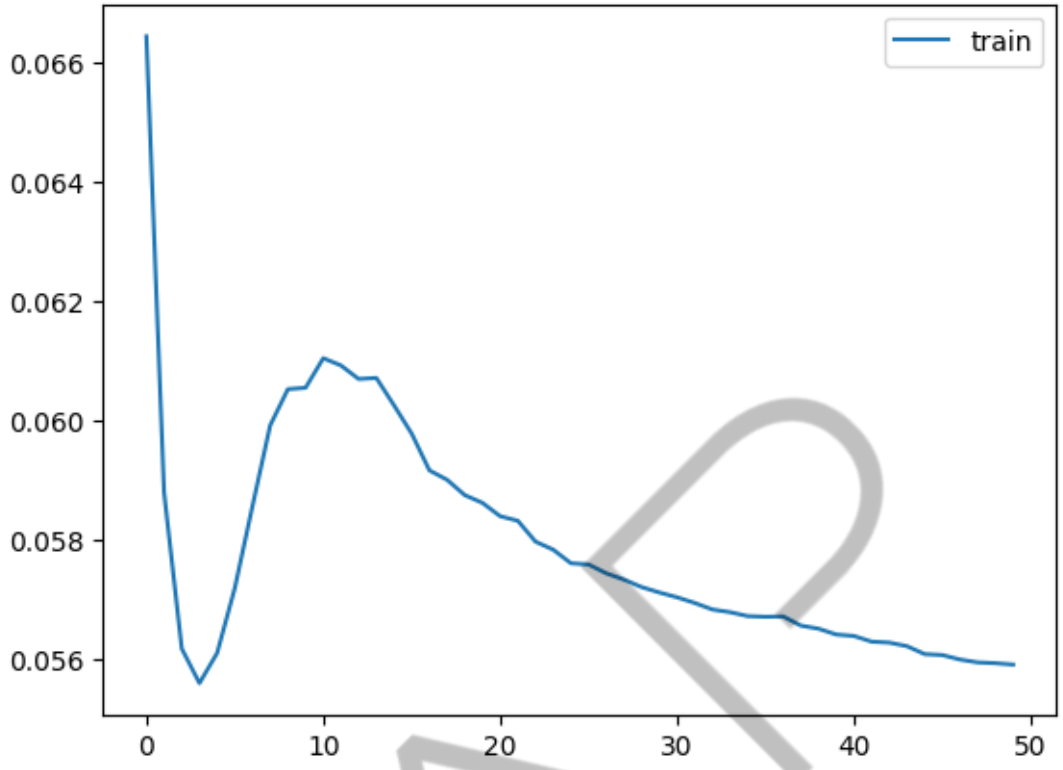


Figura 30 - Loss do treinamento
Fonte: Elaborado pelo autor (2025)

Já a figura a seguir mostra a diferença entre os valores reais e os preditos:

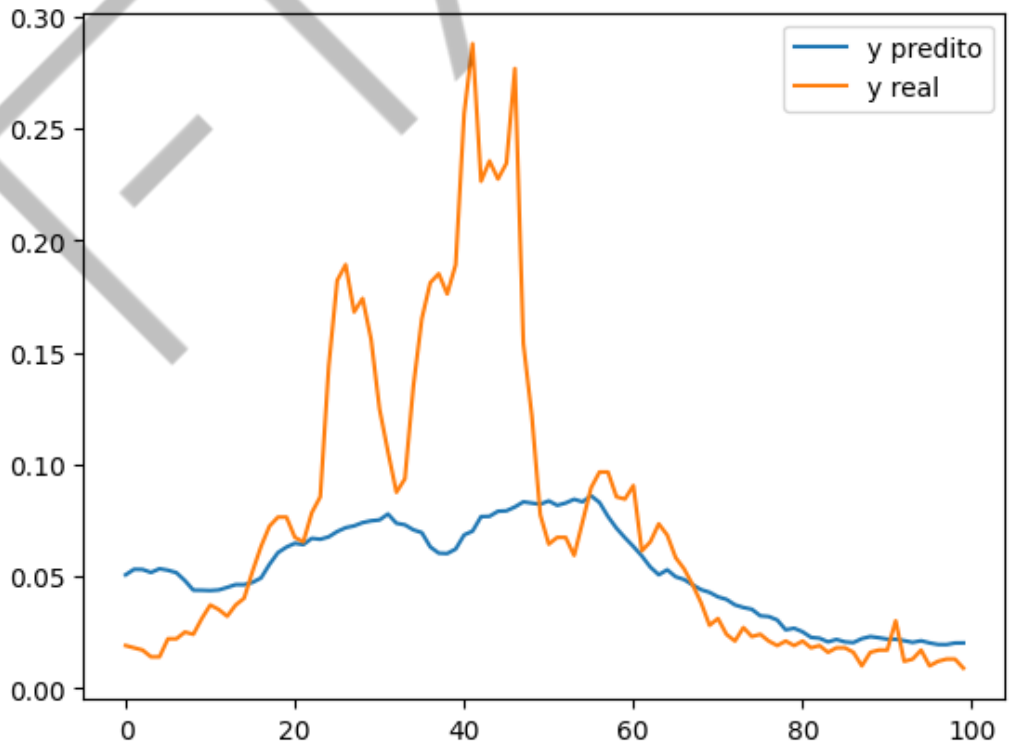


Figura 31 - Diferenças entre os valores reais e preditos
Fonte: Elaborado pelo autor (2025)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Neste caso, nosso modelo teve um RMSE: 0.095. Poderíamos melhorar adicionando mais dados ao treinamento ou ampliando o lag (dias de dados em cada observação), ou até mesmo treinando por mais tempo.

7.3 Classificação de e-mails em spam ou não (trabalhando com texto)

Já no ambiente do Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 4c), e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

Dica: para este código do capítulo, utilize o arquivo "**SPAM text message 20170820 - Data.csv**" fornecido.

Sobre a base de dados:

Esta é uma base de dados com textos de e-mails escritos em Inglês, sendo categorizados como *spam* ou *ham* (termo do meio tecnológico que designa um e-mail que não é spam). Nossa tarefa é seguir o pipeline de NLP e criar um classificador de e-mails, assim como qualquer provedor deste serviço já faz para separar as mensagens nas pastas adequadas.

Vamos para o código!

Começaremos com os *imports* e carregamento da base:

```
import pandas as pd
import matplotlib.pyplot as plt

from bs4 import BeautifulSoup
import string
import re
import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import
CountVectorizer, TfidfVectorizer
import numpy as np

from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.ensemble import RandomForestClassifier
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
from sklearn.metrics import accuracy_score, f1_score,
confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns


df = pd.read_csv('SPAM text message 20170820 - Data.csv')
print(df.shape)
```

Código-fonte 18 - Imports e leitura da base
Fonte: Elaborado pelo autor (2025)

A figura a seguir mostra a saída da leitura, mostrando apenas o tamanho do DataFrame que comporta os dados:

```
[ ] df = pd.read_csv('SPAM text message 20170820 - Data.csv')
    print(df.shape)

    (5572, 2)
```

Figura 32 - Tamanho do DataFrame
Fonte: Elaborado pelo autor (2025)

Podemos também fazer uma inspeção dos dados:

```
# Inspecionando os tipos de dados
print(df.info())

# Analisando a distribuição das classes
print()
print(df['Category'].value_counts())
print()

# Visualizando a distribuição de spam/ham
df['Category'].value_counts().plot(kind='bar')
plt.title('Distribuição de Spam/Ham')
plt.xlabel('Categoria')
plt.ylabel('Contagem')
plt.show()
```

Código-fonte 19 - Analisando a distribuição da base
Fonte: Elaborado pelo autor (2025)

A próxima figura mostra as informações gerais da base e que ela é desequilibrada, tendo mais exemplos de *hams* do que de *spams*:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 2 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Category  5572 non-null   object
 1   Message   5572 non-null   object
dtypes: object(2)
memory usage: 87.2+ KB
None

Category
ham     4825
spam     747
Name: count, dtype: int64
```

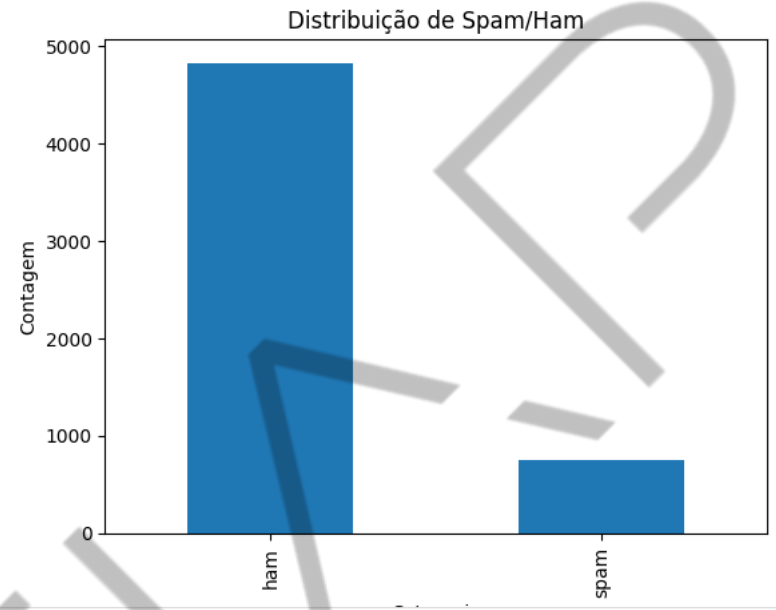


Figura 33 - Distribuição das classes na base
Fonte: Elaborado pelo autor (2025)

Se quisermos dar uma olhada nas mensagens em si, o próximo comando pode ser muito útil! Ele pede uma amostra de 5 spams e 5 hams. Note que sua saída pode ser diferente da figura “Exemplos de e-mails da base fornecida”, pois fazemos uma amostra aleatória:

```
# Explore the text data
print("Sample Spam Messages:")
print(df[df['Category'] == 'spam']['Message'].sample(5))

print("\nSample Ham Messages:")
print(df[df['Category'] == 'ham']['Message'].sample(5))
```

Código-fonte 20 - Inspecionando alguns e-mails
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
Sample Spam Messages:
1745    Someone has conacted our dating service and en...
3272    You have 1 new voicemail. Please call 08719181...
1089    You are awarded a SiPix Digital Camera! call 0...
856     Talk sexy!! Make new friends or fall in love i...
4953    Welcome to Select, an O2 service with added be...
Name: Message, dtype: object

Sample Ham Messages:
5508    Machan you go to gym tomorrow,  i wil come lat...
1267    Hey sorry I didntgive ya a a bellearlier hunny...
5249        K I'm leaving soon, be there a little after 9
780                                 That means get the door
5094    Hi Shanil,Rakhesh here.thanks,i have exchanged...
Name: Message, dtype: object
```

Figura 34 - Exemplos de e-mails da base fornecida
Fonte: Elaborado pelo autor (2025)

Agora chegou a hora de fazermos uma limpeza nos dados. No caso de texto, temos que eliminar as marcações HTML, pontuação, espaços extras etc. O código a seguir cria uma coluna com o texto limpo e verifica se alguma linha ficou vazia:

```
def clean_text(text):
  """Limpa o texto de entrada removendo HTML, pontuação e
convertendo para minúsculas."""
  if isinstance(text, str):
    text = BeautifulSoup(text, "lxml").text
    text = re.sub(r'\|\|\|', r' ', text)
    text = re.sub(r'http\S+', r'<URL>', text)
    text = text.lower()
    text = text.replace('x', '')
      text = text.translate(str.maketrans('', '',
string.punctuation))
    text = text.lower()
  return text

# Aplique a função clean_text à coluna 'Message' e crie
uma nova coluna 'Cleaned_Message'
df['Cleaned_Message'] = df['Message'].apply(clean_text)

# Verifique se há valores ausentes na coluna
'Cleaned_Message'
print(df['Cleaned_Message'].isnull().sum())
```

Código-fonte 21 - Limpando o texto
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A saída esperada da célula anterior é “0”, ou seja, sem linhas e sem dados. Feito isso, podemos preparar os dados para Machine Learning, isto é, dividir o treino e o teste:

```
# Divida os dados em conjuntos de treinamento e teste
X_train, X_test, y_train, y_test = train_test_split(
    df['Cleaned_Message'],
    df['Category'],
    test_size=0.2,
    random_state=42,
    stratify=df['Category']
)

# Preparar rótulos de destino (converter 'Categoria' em
representação numérica)
label_mapping = {'ham': 0, 'spam': 1}
y_train = y_train.map(label_mapping)
y_test = y_test.map(label_mapping)
```

Código-fonte 22 - Dividindo a base em TREINO e TESTE
Fonte: Elaborado pelo autor (2025)

O código anterior não possui saída. Podemos então passar para a etapa de engenharia de features, que no caso do processamento de texto é criar a representação numérica de cada texto via alguma técnica conhecida, como o BoW, TF-IDF ou até mesmo algum embedding:

```
# Bag-of-Words (BoW)
vectorizer_bow = CountVectorizer()
X_train_bow = vectorizer_bow.fit_transform(X_train)
X_test_bow = vectorizer_bow.transform(X_test)

# TF-IDF
vectorizer_tfidf = TfidfVectorizer()
X_train_tfidf = vectorizer_tfidf.fit_transform(X_train)
X_test_tfidf = vectorizer_tfidf.transform(X_test)

# Embeddings de palavras (usando embeddings de palavras
médios como exemplo)
# Você pode substituir isso pelo seu modelo de embedding
de palavras preferido
# (Word2Vec ou GloVe) e lidar com palavras fora do
vocabulário adequadamente.
def get_average_word_embedding(text,
word_embedding_model):
    """
    Calculates the average word embedding of a text.
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        Args:
          text: The input text string.
          word_embedding_model: A dictionary mapping words to
their embeddings.

        Returns:
          The average word embedding of the text, or a zero
vector if no words are found.
        """
        words = text.split()
          embeddings = [word_embedding_model.get(word,
np.zeros(300)) for word in words]
        if embeddings:
          return np.mean(embeddings, axis=0)
        else:
          return np.zeros(300)

      # Espaço reservado para um modelo de incorporação de
palavras pré-treinado
      word_embedding_model = {}

      X_train_embeddings =
np.array([get_average_word_embedding(text,
word_embedding_model) for text in X_train])
      X_test_embeddings =
np.array([get_average_word_embedding(text,
word_embedding_model) for text in X_test])
```

Código-fonte 23 - Criando as representações numéricas dos textos
Fonte: Elaborado pelo autor (2025)

O código anterior também não possui saída. Uma vez que os textos já estão representados de forma numérica, podemos definir e treinar nossos modelos preditivos. Para efeitos de comparação, empregaremos 4 algoritmos (Regressão Logística, SVM, Multinomial Naive Bayes e Floresta Aleatória) com cada uma das 3 representações (BoW, TF-IDF e um embedding simples), totalizando 12 modelos candidatos. A próxima célula estabelece esses modelos e os treina, gerando um dicionário de modelos treinados (a célula também não possui saída).

```
      # Modelos para representação BoW
      lr_bow = LogisticRegression(max_iter=1000)
      lr_bow.fit(X_train_bow, y_train)

      svm_bow = SVC()
      svm_bow.fit(X_train_bow, y_train)
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
nb_bow = MultinomialNB()
nb_bow.fit(X_train_bow, y_train)

rf_bow = RandomForestClassifier()
rf_bow.fit(X_train_bow, y_train)

# Modelos para representação TF-IDF
lr_tfidf = LogisticRegression(max_iter=1000)
lr_tfidf.fit(X_train_tfidf, y_train)

svm_tfidf = SVC()
svm_tfidf.fit(X_train_tfidf, y_train)

nb_tfidf = MultinomialNB()
nb_tfidf.fit(X_train_tfidf, y_train)

rf_tfidf = RandomForestClassifier()
rf_tfidf.fit(X_train_tfidf, y_train)

# Modelos para Word Embeddings
lr_embeddings = LogisticRegression(max_iter=1000)
lr_embeddings.fit(X_train_embeddings, y_train)

svm_embeddings = SVC()
svm_embeddings.fit(X_train_embeddings, y_train)

rf_embeddings = RandomForestClassifier()
rf_embeddings.fit(X_train_embeddings, y_train)

nb_embeddings = MultinomialNB()
nb_embeddings.fit(X_train_embeddings, y_train)

# Dicionário de modelos treinados
trained_models = {
    'lr_bow': lr_bow,
    'svm_bow': svm_bow,
    'nb_bow': nb_bow,
    'rf_bow': rf_bow,
    'lr_tfidf': lr_tfidf,
    'svm_tfidf': svm_tfidf,
    'nb_tfidf': nb_tfidf,
    'rf_tfidf': rf_tfidf,
    'lr_embeddings': lr_embeddings,
    'svm_embeddings': svm_embeddings,
    'rf_embeddings': rf_embeddings,
    'nb_embeddings': nb_embeddings,
}
```

Código-fonte 24 - Definindo e treinando os modelos preditivos
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Uma vez que os modelos estão treinados, podemos avaliar seus desempenhos. Para isso, a próxima célula faz as predições no TESTE, salva os resultados em termos de Acurácia, F1-Score e Matriz de Confusão, gerando um dicionário de resultados. Tal dicionário alimenta um DataFrame Pandas, já ordenado de forma decrescente da maior para a menor acurácia (figura “Resultado do treinamento do modelo”):

```
evaluation_results = {}

for model_name, model in trained_models.items():
  if 'bow' in model_name:
    y_pred = model.predict(X_test_bow)
  elif 'tfidf' in model_name:
    y_pred = model.predict(X_test_tfidf)
  elif 'embeddings' in model_name:
    y_pred = model.predict(X_test_embeddings)
  else:
    continue

  accuracy = accuracy_score(y_test, y_pred)
  f1 = f1_score(y_test, y_pred)
  conf_matrix = confusion_matrix(y_test, y_pred)

  evaluation_results[model_name] = {
      'accuracy': accuracy,
      'f1_score': f1,
      'confusion_matrix': conf_matrix,
  }

# Vizualiza os resultados como um dataframe ordenado por
acurácia
pd.DataFrame(evaluation_results).T.sort_values(by='accur
acy', ascending=False)
```

Código-fonte 25 - Avaliando o desempenho dos modelos
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	accuracy	f1_score	confusion_matrix
lr_bow	0.98296	0.9319	[[966, 0], [19, 130]]
nb_bow	0.981166	0.925267	[[964, 2], [19, 130]]
svm_tfidf	0.981166	0.924188	[[966, 0], [21, 128]]
svm_bow	0.975785	0.900369	[[966, 0], [27, 122]]
rf_tfidf	0.970404	0.875472	[[966, 0], [33, 116]]
rf_bow	0.966816	0.858238	[[966, 0], [37, 112]]
lr_tfidf	0.965022	0.849421	[[966, 0], [39, 110]]
nb_tfidf	0.95157	0.778689	[[966, 0], [54, 95]]
lr_embeddings	0.866368	0.0	[[966, 0], [149, 0]]
svm_embeddings	0.866368	0.0	[[966, 0], [149, 0]]
rf_embeddings	0.866368	0.0	[[966, 0], [149, 0]]
nb_embeddings	0.866368	0.0	[[966, 0], [149, 0]]

Figura 35 - Resultado do treinamento do modelo
Fonte: Elaborado pelo autor (2025)

A figura anterior mostra que o melhor modelo, para este conjunto de dados, foi a regressão logística treinada sobre uma representação BoW. Vamos comparar o melhor com o pior modelo, plotando as matrizes de confusão:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     # Escolhendo o MELHOR e o PIOR modelo para plotar a matriz
de confusão
     melhor = "lr_bow"
     pior = "svm_embeddings"

     def plot_confusion_matrix(results, model_name):
       conf_matrix = results['confusion_matrix']
       plt.figure(figsize=(6, 4))
          sns.heatmap(conf_matrix,    annot=True,    fmt='d',
cmap='Blues',
                                xticklabels=['Ham',   'Spam'],
yticklabels=['Ham', 'Spam'])
       plt.xlabel('Predicted Label')
       plt.ylabel('True Label')
       plt.title(f'Confusion Matrix for {model_name}')
       plt.show()

     plot_confusion_matrix(evaluation_results[melhor],
melhor)
     print()
     plot_confusion_matrix(evaluation_results[pior], pior)
```

Código-fonte 26 - Comparando o melhor e o pior modelo
Fonte: Elaborado pelo autor (2025)

A figura “Matriz de confusão do melhor e pior classificador de e-mails” mostra que o melhor modelo realmente aprendeu a diferença entre spams e hams, e também conseguiu classificar corretamente a maior parte das mensagens. Apenas 19 delas foram classificadas incorretamente, ou seja, o classificador apontou o e-mail como “válido” deixando-o na caixa de entrada, quando na verdade era um spam. Quem nunca passou por isso, não é mesmo?

Por outro lado, o pior modelo não aprendeu nada! Ele simplesmente chutou que todos os e-mails eram válidos e deixou todos na caixa de entrada! Portanto, não teve aderência alguma ao problema.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

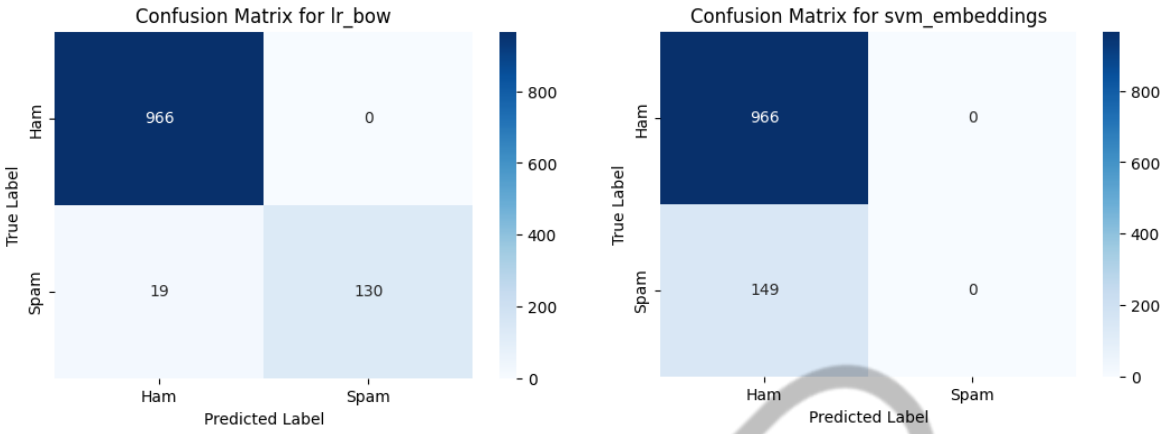


Figura 36 - Matriz de confusão do melhor e pior classificador de e-mails
Fonte: Elaborado pelo autor (2025)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

Ao longo deste capítulo, exploramos RNNs, LSTMs e GRUs, compreendendo a sua importância na modelagem de dados sequenciais, e como essas arquiteturas lidam com dependências temporais de curto e longo prazo. Estudamos a diferença entre abordagens univariadas e multivariadas, entendemos como transformar séries temporais em dados supervisionados e discutimos a relevância de ajustar o formato da entrada para modelos de Deep Learning. Além disso, destacamos a aplicação dessas redes no PLN, permitindo a modelagem de texto para tradução automática, chatbots e análise de sentimentos.

As aplicações dessas redes são vastas e impactam diversas áreas do conhecimento. No mercado financeiro, as LSTMs são usadas para prever flutuações no preço de ações e identificar padrões de investimento. Na previsão meteorológica, redes recorrentes analisam séries temporais para antecipar mudanças climáticas com maior precisão. No reconhecimento de fala, modelos treinados com sequências de áudio possibilitam o desenvolvimento de assistentes virtuais, como Alexa e Google Assistant, que compreendem e respondem à fala humana em tempo real.

O avanço das redes neurais recorrentes abriu caminho para o uso de modelos ainda mais poderosos, como os transformers, que revolucionaram o Processamento de Linguagem Natural. No entanto, as RNNs e suas variantes ainda são fundamentais para muitas aplicações práticas, especialmente onde a sequência dos dados desempenha um papel crítico.

O aprendizado de modelos sequenciais é um desafio empolgante, que exige preparação cuidadosa dos dados, escolha adequada dos hiperparâmetros e uma abordagem experimental para encontrar a melhor arquitetura. À medida que avançamos para técnicas mais sofisticadas, entender essas redes neurais nos dá uma base sólida para construir sistemas mais robustos e inteligentes. O conhecimento adquirido aqui será essencial para enfrentar desafios reais em ciência de dados, IA e Aprendizado de Máquina.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

AGRAWAL, Nipun. Sentence Correction Using Recurrent Neural Network. **Medium**, 15 de abril de 2021. Disponível em: <https://medium.com/geekculture/sentence-correction-using-recurrent-neural-network-6321527ee08b>. Acesso em: 19 mar. 2025.

ALI, Moez. Tutorial de análise de sentimentos com NLTK para iniciantes. **DataCamp**, 07 de agosto de 2024. Disponível em: <https://www.datacamp.com/pt/tutorial/text-analytics-beginners-nltk>. Acesso em: 19 mar. 2025.

BATISTA, Ivanildo. Series-Temporais. **GitHub**, 2022. Disponível em: <https://github.com/IvanildoBatista/Series-Temporais>. Acesso em: 19 mar. 2025.

BENGIO, Yoshua; GOODFELLOW, Ian; COURVILLE, Aaron. **Deep learning**. Cambridge, MA, USA: MIT press, 2017.

CHURCH, Kenneth Ward. Word2Vec. **Natural Language Engineering**, 2017, 23.1: 155-162.

DEJESUS, Thiago. Tendência, Sazonalidade e Resíduos: Séries Temporais com o Python. **Medium**, 11 de fevereiro de 2024. Disponível em: <https://medium.com/@thiago.dejesus0705/tend%C3%AAncia-sazonalidade-e-res%C3%ADduos-s%C3%A9ries-temporais-com-o-python-5e19787ce07e>. Acesso em: 19 mar. 2025.

GATED Recurrent Units (GRU). **Dive into Deep Learning**, 2025. Disponível em: <https://pt.d2l.ai/chapter_recurrent-modern/gru.html>. Acesso em: 19 mar. 2025.

GÉRON, Aurélien. **Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow**. O'Reilly Media, Inc., 2022.

IQBAL, Touseef; SAMBYAL, Abhishek Singh; PADHA, Devanand. A Review of Text Summarization using Gated Neural Networks. **ResearchGate**, abril de 2018. Disponível em: <https://www.researchgate.net/figure/Figure-of-unfolded-RNN_fig1_328281924>. Acesso em: 19 mar. 2025.

MEMÓRIA Longa de Curto Prazo (LSTM). **Dive into Deep Learning**, 2025. Disponível em: <https://pt.d2l.ai/chapter_recurrent-modern/lstm.html>. Acesso em: 19 mar. 2025.

PENNINGTON, Jeffrey; SOCHER, Richard; MANNING, Christopher D. Glove.: **Global vectors for word representation**. In: Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP). 2014. p. 1532-1543.

QADER, Wisam A.; AMEEN, Musa M.; AHMED, Bilal I. **An overview of bag of words;** importance, implementation, applications, and challenges. In: 2019 international engineering conference (IEC). IEEE, 2019. p. 200-204.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

RAM, Vijay. Word2vec - Word embeddings used in NLP. **LinkedIn**, 18 de fevereiro de 2018. Disponível em: <https://www.linkedin.com/pulse/word2vec-word-embeddings-used-nlp-vijay-ram/>. Acesso em: 19 mar. 2025.

ROSSI, Rafael Geraldeli; FARIAS, Thiago S. Arquitetura da Rede Bi-LSTM. **ResearchGate**, julho de 2021. Disponível em: <https://www.researchgate.net/figure/Figura-9-Arquitetura-da-Rede-Bi-LSTM_fig5_353347502>. Acesso em: 19 mar. 2025.

SHERSTINSKY, Alex. **Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network**. Physica D: Nonlinear Phenomena, 2020, 404: 132306.

SPARCK, Karen Jones. A statistical interpretation of term specificity and its application in retrieval. **Journal of documentation**, 1972, 28.1: 11-21.

VASWANI, Ashish, *et al*. Attention is All You Need. **Advances in neural information processing systems**, 2017, 30.

YAO, Tengjun; ZHAI, Zhengang; GAO, Bingtao. **Text classification model based on fasttext**. In: 2020 IEEE International conference on artificial intelligence and information systems (ICAIIS). IEEE, 2020. p. 154-157.

YU, Yong, *et al*. A review of recurrent neural networks: LSTM cells and network architectures. **Neural computation**, 2019, 31.7: 1235-1270.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

Termo	Explicação
BERT	Bidirectional Encoder Representations from Transformers.
BoW	Bag of Words.
BPTT	Backpropagation Through Time — Retroalimentação dependente do tempo.
CBOW	Continuous Bag of Words.
CNN	Convolutional Neural Network — Redes Neurais Convolucionais.
FAIR	Facebook AI Research.
GloVe	Global Vectors for Word Representation.
GPT	Generative Pre-trained Transformer.
GRU	Gated Recurrent Unit — Unidade Recorrente Fechada.
IDF	Frequência Inversa de Documentos.
LSTM	Long Short-Term Memory — (Rede Neural de) Memória de Curto e Longo Prazo.
NLP	Natural Language Processing — Processamento de Linguagem Natural.
RNNs	Recurrent Neural Networks — Redes Neurais Recorrentes.
TF	Frequência de Termos.
TF-IDF	Term Frequency-Inverse Document Frequency.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com